

AAA

Add argument to A

Opcode:

172400 (octal) = 0xF500

Format:

AAA <number>

Mask:

1111_1111_0000_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								number							

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
number	7-0	numeric	8-bit argument extended to 16 bits using sign extension

Flags: C, O

Examples

Add 3 to the contents of the A register.

```
AAA 3
```

AAB

Add argument to B

Opcode:

172000 (octal) = 0xF400

Format:

AAB <number>

Mask:

1111_1111_0000_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								number							

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
number	7-0	numeric	8-bit argument extended to 16 bits using sign extension

Flags: C, O

Examples

Add -26 to the contents of the B register.

AAB -26

AAT

Add argument to T

Opcode:

173000 (octal) = 0xF600

Format:

AAT <number>

Mask:

1111_1111_0000_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								number							

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
number	7-0	numeric	8-bit argument extended to 16 bits using sign extension

Flags: C, O

Examples

Add 13 to the contents of the T register.

AAT 13

AAX

Add argument to X

Opcode:

173400 (octal) = 0xF700

Format:

AAX <number>

Mask:

1111_1111_0000_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								number							

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
number	7-0	numeric	8-bit argument extended to 16 bits using sign extension

Flags: C, O

Examples

Add 5 to the contents of the X register.

```
AAX 5
```

ADD

Add to A register

$A \leftarrow A + (EL)$
Add the contents of the memory location pointed to by the effective address to the A register, leaving the result in A.

Opcode:	060000 (octal) = 0x6000
Format:	ADD <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

Flags: C, O, Q

ADDD

Add two decimal operands.

$(op1) \leftarrow (op1) + (op2)$

Add the second operand to the first operand, leaving the result in the first operand's location.

If the first operand field is too short to contain all the significant digits of the result, then decimal overflow occurs.

If bit 13 of D2 in the first operand is set, the sign of the result will be 17₈ (BCD unsigned).

Any empty operand, that is with a field length of zero, is treated as a positive zero.

****Instruction sequence:****

ADDD
error handling instruction
next instruction after ADDD or after error handling routine

Note:
Operands should be normalized before this instruction is executed using the SHDE instruction.

****Example:****

ADDD
JMP *12₈
STX 20₈

The ADDD instruction causes the program counter to skip the next instruction UNLESS an error has been generated. In this case, the instruction immediately after ADDD will handle the error in some way (in this example a jump is executed on error to ((P) + 12₈)).

(* is the assembler mnemonic for the P register)

Opcode:

0140120 (octal) = 0xC050

Format:

ADDD

Mask:

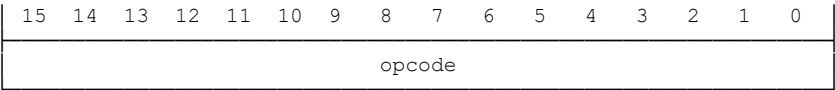
1111_1111_1111_1111

Privilege:

User

Class:

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

AND

Logical AND to A register
 $A \leftarrow A \& (EA)$

Perform a bitwise AND operation between the contents of the A register and the contents of the memory location pointed to by the effective address, leaving the result in A.

Opcode:	070000 (octal) = 0x7000
Format:	AND <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

BANC

logical AND with bit complement

$$K \leftarrow K \& (B)0$$

Perform a bitwise AND with the contents of the B register and the contents of the K register, leaving the result in K.

Opcode:	177000 (octal) = 0xFE00
Format:	BANC
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Flags: K

BAND

Logical AND to K

$K \leftarrow K \& (B)$

Opcode:	177200 (octal) = 0xFE80
Format:	BAND
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Flags: K

BFILL

Byte fill.

Only the destination is used as an operand in this instruction (it is placed in the X and T registers). The lower byte of the A register is then filled with the destination field.

After execution, bit 15 of the T register points to the end of the field (after the last byte position) and the field length equals zero.

Opcode:	0140130 (octal) = 0xC058
Format:	BFILL
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

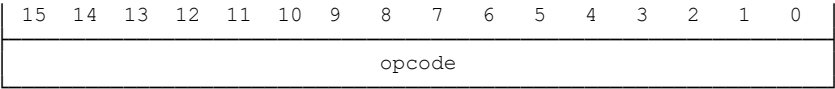
BLDA

Load K

$K \leftarrow \text{bit}$

Opcode:	176600 (octal) = 0xFD80
Format:	BLDA
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	

Bit Format



Flags: K

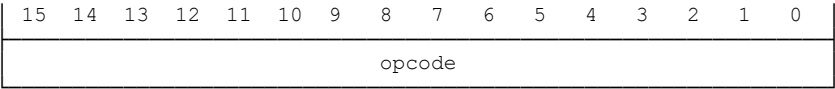
BLDC

Load K and complement

$K \leftarrow (B) \text{ (complemented)}$

Opcode:	176400 (octal) = 0xFD00
Format:	BLDC
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	

Bit Format



Flags: K

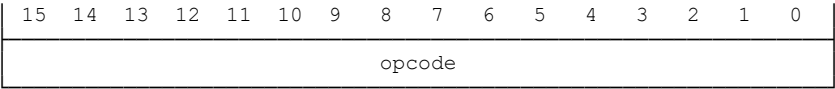
BORA

Logical OR to K

$K \leftarrow K \mid (B)$

Opcode:	177600 (octal) = 0xFF80
Format:	BORA
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	

Bit Format



Flags: K

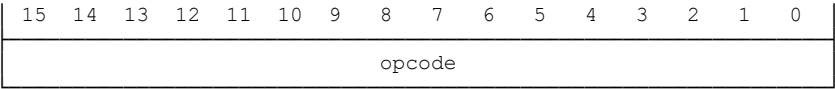
BORC

Logical OR with bit complement

$$K \leftarrow K \mid (B)0$$

Opcode:	177400 (octal) = 0xFF00
Format:	BORC
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	

Bit Format



Flags: K

Examples

Complement bit 6 in the X register, then OR the bit with K, leaving the result in K

```
BORC 60 DX
```

BSET

Set specified bit in equal to specified condition

Opcode:

174000 (octal) = 0xF800

Format:

BSET <condition> <bit_no> <destination>

Mask:

1111_1111_1000_0000

Privilege:

User

Class:

bit_instructions_with_condition

Bit Format

15	14	13	12	11	10	9	8	7	7	6	5	4	3	2	1	0
opcode					condition				bit_no					destination		

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
condition	10-7	enum	Specify condition for BSKP and BSET
bit_no	7-3	numeric	Specify bit number for <dsestination> = STS
destination	2-0	enum	Destination register (dr)

Examples

Reset the static overflow flag

```
BSET ZRO SSO
```

BSKP

Skip next location if specified condition is true

$P \leftarrow P+1$

Opcode:	175000 (octal) = 0xFA00
Format:	BSKP <condition> <bit_no>
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	bit_instructions_with_condition

Bit Format

15	14	13	12	11	10	9	8	7	7	6	5	4	3	2	1	0
opcode					condition				bit_no					destination		

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
condition	10-7	enum	Specify condition for BSKP and BSET
bit_no	7-3	numeric	Specify bit number for <dsestination> = STS
destination	2-0	enum	Destination register (dr)

Examples

Skip the next location if the cary flag is set

```
BSKP ONE SSC
```

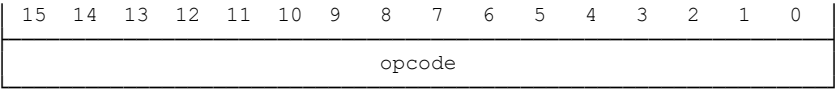
BSTA

Store and clear K

$(B) \leftarrow K; K \leftarrow 0$

Opcode:	176200 (octal) = 0xFC80
Format:	BSTA
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	

Bit Format



Flags: K

BSTC

Store complement and set K to 1

$(B) \leftarrow K \text{ (complemented)}; K \leftarrow 1$

Opcode:	176000 (octal) = 0xFC00
Format:	BSTC
Mask:	1111_1111_1000_0000
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Flags: K

CHREENTPAGES

Change page tables.

The X register is used to address the current (R1) and previous (Rp) scratch registers.

If the R1 is zero, the reentrant page has nothing to change so the loop is left, otherwise the contents of the memory location pointed to by the R1 + 2 are loaded into T.

T then contains the protect table entry, if the page has not been written to (WIP bit 12 is zero) T and R1 are loaded with Rp. R1 (now containing Rp) is tested again for zero. If the page has been written to, the T register is loaded with the contents of the second scratch register (R2), pointed to by R1, and R2 becomes the address of Rp. X is loaded with R1 as the new pointer to the reentrant pages and Rp is loaded into the D register pointed to by A.

Opcode:

140303 (octal) = 0xC0C3

Format:

CHREENTPAGES

Mask:

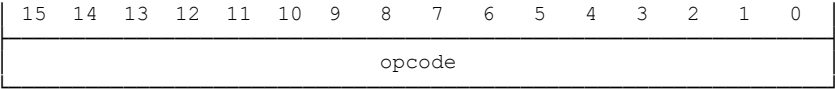
1111_1111_1111_1111

Privilege:

Privileged

Class:

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

CLEPT

Clear page tables.

Opcode:	140301 (octal) = 0xC0C1
Format:	CLEPT
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Description

This instruction can replace the following instructions:

```
CLEPT: JXZ *108
      LDBTX 108
      LDA ,B
      JAZ *38
      STATX 208
      STZ ,B
      LDXTX 00
      JMP *-78
```

Each time the loop is executed (until X becomes zero) the physical memory location addressed by X is loaded into the B register.

The B register contents provide the address of a page table entry, which is loaded into the A register.

If the page table entry is zero (unused) the loop is restarted.

If the page table entry is not zero (used) it is stored in a physical location addressed by X (8 locations away from its original entry) and the original page table entry cleared by placing zero in the location addressed by the B register.

The physical location addressed by X is then loaded into the X register itself and the loop restarted.

* is the mnemonic for P relative addressing.

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

CLEPU

Clear page tables and collect PGU information.

Opcode:

140304 (octal) = 0xC0C4

Format:

CLEPU

Mask:

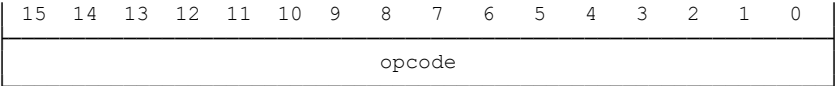
1111_1111_1111_1111

Privilege:

Privileged

Class:

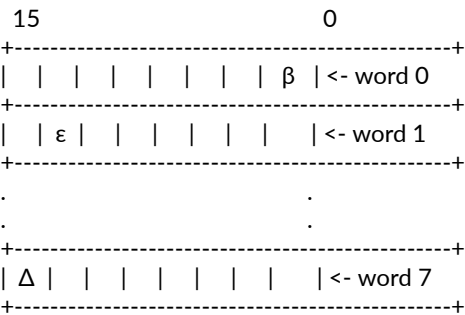
Bit Format



Description

This instruction collects information on the PGU (page used) bit of a page table entry whilst executing CLEPT.

The instruction places PGU information in an eight word table called the page map bank. Each bit in the bank represents the status of a page's PGU bit as follows:



Δ denotes page 177₈ PGU bit
ε denotes page 32₈ PGU bit
β denotes page 0₈ PGU bit

The L register contains the address of the map entry.

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

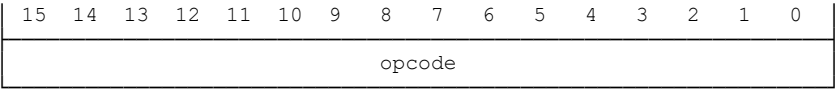
CLNREENT

Clear non reentrant pages.

The contents of the memory address at A + 2 are read to find the page table to be cleared along with the SINTRAN RT bitmap (addressed by the X and T registers).
The page table entries corresponding to those bits set in the RT bitmap are then cleared.

Opcode:	140302 (octal) = 0xC0C2
Format:	CLNREENT
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

CLPT

Clear segment from the page tables.

Opcode:	140505 (octal) = 0xC145
Format:	CLPT
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

CNREK

Clear non reentrant pages (SINTRAN K only).

Opcode:	140504 (octal) = 0xC144
Format:	CNREK
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

COMD

Compare two decimal operands.

$(A) \leftarrow (op1) \text{ compared to } (op2)$

$(A) = 0$ if $(op1) = (op2)$

$(A) = 1$ if $(op1) > (op2)$

$(A) = -1$ if $(op1) < (op2)$

Compare the first operand with the second operand, leaving the result in the A register.

If the two operands are unequal in field length, the shorter operand is extended with zeros to allow comparison. The operands are unaffected by the instruction.

The positions of the decimal points are not taken into account when the two operands are compared, so the two operands should be normalized using the SHDE instruction first.

Any empty operand, that is with a field length of zero, is treated as a positive zero. An unsigned number is treated as positive. Positive and negative zeros are equal.

****Instruction sequence:****

COMD

error handling instruction

next instruction after COMD or after error handling routine

****Example:****

COMD

JMP *30₈

AAA 20₈

The COMD instruction causes the program counter to skip the next instruction UNLESS an error has been generated. In this case, the instruction immediately after COMD will handle the error in some way (in this example a jump is executed on error to $((P) + 30_8)$).

(* is the assembler mnemonic for the P register)

Opcode:	0140122 (octal) = 0xC052
Format:	COMD
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

COPY

Copy source to destination

COPY is a compound mnemonic for RADD CLD.

Optional subinstructions:
The following sub-instructions are allowed:
CM1, CM2, ADC, AD1

Note:
Using ADC and AD1i in the same instruction generates a no-operation.

Opcode:	0146100 (octal) = 0xCC40
Format:	COPY [sub-instruction(s)] <source> <destination>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	register_src_dst

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode										source			destination		

Operands

Name	Bits	Type	Description
opcode	15-6	opcode	The opcode for register arithmetic
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

Copy the contents of the A register to the D register.

```
COPY SA DD
```

2's complement the A register. (Equivalent to RADD CM1 ADC SA DA)

```
COPY CM2 SA DA
```

DEPO

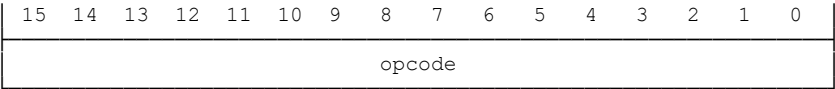
Deposit

Store the contents of the T register in the physical memory location pointed to by the A and D register contents.

Opcode:
Format:
Mask:
Privilege:
Class:

150417 (octal) = 0xD10F
DEPO
1111_1111_1111_1111
Privileged

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

DNZ

Denormalise
Convert the floating number in TAD to a fixed point number in A

Opcode:

152000 (octal) = 0xD400

Format:

DNZ <scaling_factor>

Mask:

1111_1111_0000_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								scaling_factor							

Description

The number in the floating point accumulator (A and D registers) is converted to its Single precision fixed point equivalent in the A register using the scaling factor given.

When converting to an integer, a scaling factor of -16 should always be used and will give a fixed point number with the same value as the integer part of the floating point number.
Other scaling factors will have the same result but the overflow test will be affected.

The D register will be cleared after this instruction.

If the conversion causes underflow, the A and D registers will be set to zero. If overflow occurs (the resulting integer has an absolute value greater than decimal 32767), the error flag (Z) is set to one.

48-bit:
* 48-bit CPUs allow different scaling factors to be used for DNZ operations. However, the overflow test is only failproof for a scaling factor of 16 (decimal)

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode for floating conversion
scaling_factor	7-0	numeric	Scaling factor in range -128 to 127 (gives converting range from 10^-39 to 10^39)

Examples

Convert the number in TAD to a fixed point number in A

DNZ

Floating to integer conversation. Opcode 152360

DNZ -20

ELEAV

Error leave stack.

If an error occurs, leave the stack.

This instruction saves the previous stack pointer in LINK and restores the B register to its previous value (PREVB) before leaving the stack. The stack is left by loading the P register (program counter) with the return address (LINK). The A register is loaded with an error code which is saved in the ERRCODE stack entry (pointed to by B = 173₈).

```
(B = 2008) ← (B = 2008) - 1
(P)      ← (B = 2008) {LINK}
(B)      ← (B = 1778) {PREVB}
(A)      ← ERRCODE8
(B = 1738) ← (A)      {ERRCODE}
```

Format:
ELEAV

Opcode:

0140137 (octal) = 0xC05F

Format:

ELEAV

Mask:

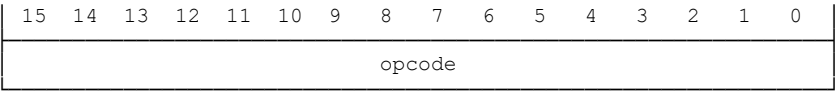
1111_1111_1111_1111

Privilege:

User

Class:

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

ENPT

Enter segment in page tables.

Opcode:

140506 (octal) = 0xC146

Format:

ENPT

Mask:

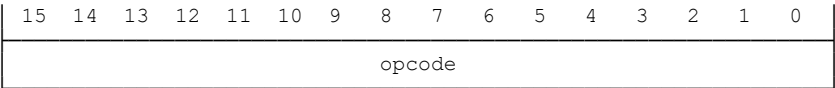
1111_1111_1111_1111

Privilege:

Privileged

Class:

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

ENTR

Enter stack.

This instruction saves the current stack pointer (B), the return address (LINK), and previous stack pointer (PREVB). It transfers the top of stack address (SMAX) and establishes the new stack demand and pointer.

```
(B = 1778) ← (B)           {save current pointer in PREVB}  
(B = 1758) ← (B = 1758)     {SMAX}  
(B = 2008) ← (L) + 1       {save return address in LINK}  
(B)      ← (B = 1768) + 2008 {new pointer}  
(B = 1768) ← stack demand + (B)
```

Stack overflow causes an error return, that is the program continues at the address following the stack demand value. In all other cases, the program skips this address to find the return address from the stack.

Format:
ENTR
<stack demand-value in words>
<error return address>
<return address>

Opcode:

0140135 (octal) = 0xC05D

Format:

ENTR

Mask:

1111_1111_1111_1111

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

Flags: PTM

EXAM

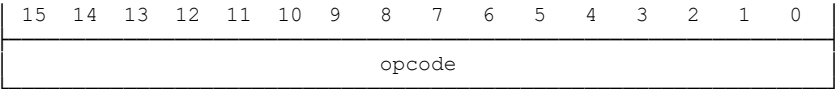
Examine

Load the contents of the physical memory location, pointed to by the A and D register contents, into the T register.

Opcode:
Format:
Mask:
Privilege:
Class:

150416 (octal) = 0xD10E
EXAM
1111_1111_1111_1111
Privileged

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

EXIT

Return from subroutine

This compound mnemonic represents the specific COPY instruction:
COPY SL DP

It causes the return from a subroutine by copying the stored return address into the program counter (P register).

Opcode:	0146142 (octal) = 0xCC62
Format:	EXIT
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

Flags: C, O, Q

EXR

Execute instruction found in specified register

The contents of the sr register are executed as the next instruction.
If sr contains a memory reference instruction, the address is given as part of the instruction.

EXR <sr> cannot be used to fetch an <sr> register containing another EXR instruction.
If you attempt this the error flag (Z) is set.

Opcode:

0140600 (octal) = 0xC180

Format:

EXR <sr>

Mask:

1111_1111_1100_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode													register		

Operands

Name	Bits	Type	Description
opcode	15-3	opcode	The opcode for execute instruction
register	2-0	enum	Register containing instruction to execute

Examples

[(A) = 014177, STX *177] Execute the instruction held in the A register. Store the K register contents in the memory location pointed to by the program counter plus 177.

EXR SA

[(B) = 134020, JPL 20] Execute the instruction held in the B register. The instruction is 4a jump to a subroutine at memory location ({P} + 20.). The return address (the address of the instruction after EXR) returned to once the subroutine has been completed is heid in the L register.

EXR SB

FAD

Add to floating point accumulator

$(A) \leftarrow (ea) + (T)$
 $(D) \leftarrow (ea + 1) + (A)$

The contents of two sequential memory locations, pointed to by the effective address, are added to the contents of the floating point accumulator (T and A registers). The result is held in the accumulator.

Opcode:

0100000 (octal) = 0x8000

Format:

FAD <addressing_mode> <disp>

Mask:

1111_1000_0000_0000

Privilege:

User

Class:

memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

FDV

Divide floating point accumulator

The contents of the floating point accumulator (A and D registers) are divided by the contents of two sequential memory locations, pointed to by the effective address.

Opcode:

114000 (octal) = 0x9800

Format:

FDV <addressing_mode> <disp>

Mask:

1111_1000_0000_0000

Privilege:

User

Class:

memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Description

- Flags affected:
- Division by zero sets the error flag (2).
 - This can be detected by the BSKP instruction

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

Flags: Z

FMU

Multiply floating point accumulator.

The contents of the floating point accumulator (A and D registers) are multiplied by the contents of two sequential memory locations, pointed to by the effective address. The result is held in the accumulator.

Opcode:

110000 (octal) = 0x9000

Format:

FMU <addressing_mode> <disp>

Mask:

1111_1000_0000_0000

Privilege:

User

Class:

memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

Flags: Z

FSB

Subtract from floating point accumulator.

The contents of two sequential memory locations, pointed to by the effective address, are subtracted from the contents of the floating point accumulator (A and D registers).
The result is held in the accumulator.

Opcode:	104000 (octal) = 0x8800
Format:	FSB <addressing_mode> <disp>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

Flags: Z

GECO

GECO is a customer-specified instruction which appears to be included as part of the standard instruction set from ND-100/CE and later.

The name comes from the customer, GECO (Geophysical Company of Norway)

SINTRAN III version L, and probably version K and possibly earlier, tests for GECO as part of the startup.

The instruction is found in the ND-110 microcode.

Opcode:	0142700 (octal) = 0xC5C0
Format:	GECO
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

IDENT

Transfer IDENT code of interrupting device with highest priority on the specified level to A register.

Opcode:

0143600 (octal) = 0xC780

Format:

IDENT <level_code>

Mask:

1111_1111_1100_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode										level_code					

Operands

Name	Bits	Type	Description
opcode	15-6	opcode	The opcode determines what type of operation occurs
level_code	5-0	enum	The interrupt level code

INIT

Initialize stack.

Loads the addresses pointed to by B with the stack frame addresses.

Sets up

```
LINK ← L + 1           {stack start}
PREVB ← (B)             {save current pointer}
SMAX ← stack start address + maximum stack size
(B) ← (B = 2008) + 2008 {establish new pointer}
STP ← stack demand + (B)
```

Load the addresses pointed to by ****B**** with the stack frame addresses.

Stack overflow and flag error causes an error return, that is the program continues at the address following the stack demand value.

In all other cases, the program skips this address to find the return address from the stack.

Format:

```
INIT
<number of words allocated to stack>
<address of stack start>
<maximum stack size>
<flag>
<address left empty>
<error return address>
<return address>
```

Opcode:

0140134 (octal) = 0xC05C

Format:

INIT

Mask:

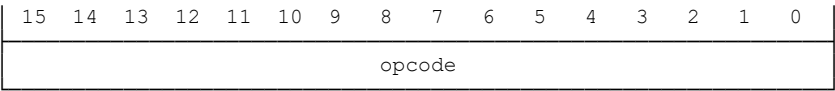
1111_1111_1111_1111

Privilege:

User

Class:

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

Flags: PTM

INSPL

Insert page in page list. (See Appendix B for a software description.)

Opcode:	140502 (octal) = 0xC142
Format:	INSPL
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

IOF

Interrupt System OFF

Disables the interrupt system.
On IOF the ND-110 continues operation at the same program level.

Opcode:	150401 (octal) = 0xD101
Format:	IOF
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

ION

Interrupt System ON

Enables the interrupt system. On ION the ND-i10 resumes operation in the program level with highest priority.

Opcode:	150402 (octal) = 0xD102
Format:	ION
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

IOT

NORD-1 INSTRUCTION (DO NOT USE)

Opcode:	160000 (octal) = 0xE000
Format:	IOT <device_register_address>
Mask:	1111_1000_0000_0000
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Description

ND-110 microcode implements this instruction like this:

1861 000627 % IOT - INSTRUCTION IS PRIVILEGED WHEN RING = 0 OR 1
1862 000627 % AND ILLEGAL WHEN RING = 2 OR 3
1863 000627
1864 000627 IOT1: ALUD,NONE IDBS,ALU T,JMP T,PUSH PVCHK;
1865 000630 ALUD,NONE IDBS,ALU T,JMP T,HOLD ILLIN;

IOX

Exchange information between I/O system and A register.

Opcode:	0164000 (octal) = 0xE800
Format:	IOX <device_register_address>
Mask:	1111_1000_0000_0000
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	10	9	8	7	6	5	4	3	2	1	0
opcode						device_register_address										

Description

This instruction is PRIVILEGED and only available to:

- programs running in system mode (rings 2-3)
- programs running without memory protection

The instruction controls all transfers between the ND-110 and any external devices.

IOX can be used to address a maximum of 2048 device registers for external devices connected to the ND-110 CPU. Data, control, and status between device and CPU can be exchanged.

The transfer direction is determined by the least significant bit of the device register address:

- If 0: input (from device to CPU)
- If 1: output (from CPU to device)

Register addresses can hold data, command, or status information for a device.

An external device may require more than one register address (e.g., a magnetic tape unit may need several).

These should be given successive device-register addresses (odd for input, even for output).

Note: The number of external devices that can be controlled by the CPU depends on the configuration of the devices.

Operands

Name	Bits	Type	Description
opcode	15-10	opcode	The opcode determines what type of operation occurs (fixed code 11101 ₂)
device_register_address	10-0	displacement	11-bit field limiting the number of external devices that can be addressed by the CPU. Bit 0 gives the direction of transfer: - 0: input (from device to CPU) - 1: output (from CPU to device)

Examples

Read status of console device into A register

```
IOX 0306
```

Write character in A register to console device

```
IOX 0305
```

IOXT

Exchange information between I/O system and A register.

Opcode: 150415 (octal) = 0xD10D
Format: IOXT
Mask: 1111_1111_1111_1111
Privilege: Privileged
Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Description

This instruction is PRIVILEGED and only available to:

- programs running in system mode (rings 2-3)
- programs running without memory protection

The instruction controls all transfers between the ND-110 and external devices.

IOXT can be used to address a maximum of 65536 device-register addresses for external devices connected to the ND-110 CPU.

The 16-bit T register holds the device register address, allowing a much larger address space than IOX. Data, control, and status between device and CPU can be exchanged.

The transfer direction is determined by the least significant bit of the device register address:

- If 0: input (from device to CPU)
- If 1: output (from CPU to device)

Instruction format:

LDT <address mode> <disp>
 IOXT

LDT instruction:

The IOXT instruction uses the T register contents as the device register address. The 16-bit T register gives a limit of 65536 register addresses.

This address MUST be loaded into the T register before IOXT is executed, hence LDT is used. (See Memory Transfer Instructions, LOAD for explanation of LDT.)

IOXT instruction:

IOXT is used as a single mnemonic.

Note: The number of external devices that can be controlled by the CPU depends on the configuration of the devices.

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

IRR

Inter Register Read
A: = specified register on specified level

Opcode:

153600 (octal) = 0xD780

Format:

IRR <level> <register>

Mask:

1111_1111_1000_0000

Privilege:

Privileged

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode									level				register		

Operands

Name	Bits	Type	Description
opcode	15-7	opcode	The opcode for inter-register read
level	6-3	uint4	Privilege level to access (0-15)
register	2-0	enum	Register to read from specified level

Examples

Copy the program counter on program level 14 into the A register of the current program level.

IRW

Inter Register Write
Write A to specified register on specified level

Opcode:

153400 (octal) = 0xD700

Format:

IRW <level> <register>

Mask:

1111_1111_1000_0000

Privilege:

Privileged

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode									level				register		

Operands

Name	Bits	Type	Description
opcode	15-7	opcode	The opcode for inter-register read
level	6-3	uint4	Privilege level to access (0-15)
register	2-0	enum	Register to read from specified level

Examples

Copy the A register on the current program level into the B register on program level 8

JAF

Condition: Jump if (A) != 0 (jump if A filled)

Opcode:	0131400 (octal) = 0xB300
Format:	JAF <displacement>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	jump_on_condition

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								displacement							

Description

If condition true, jump to the address of the program counter plus the value of displacement.
If condition false, continue program execution at (P) + 1.

'displacement' = The eight displacement (disp) bits give a signed range of -128 to 127 locations to be jumped if the condition is true.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JAN

Condition: Jump if (A) < 0 (jump if A is negative)

Opcode:	0130400 (octal) = 0xB100
Format:	JAN <displacement>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	jump_on_condition

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								displacement							

Description

If condition true, jump to the address of the program counter plus the value of displacement.
If condition false, continue program execution at (P) + 1.

'displacement' = The eight displacement (disp) bits give a signed range of -128 to 127 locations to be jumped if the condition is true.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JAP

Condition: Jump if (A) > 0 (jump if A positive)

Opcode:	0130000 (octal) = 0xB000
Format:	JAP <displacement>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	jump_on_condition

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								displacement							

Description

If condition true, jump to the address of the program counter plus the value of displacement.
If condition false, continue program execution at (P) + 1.

'displacement' = The eight displacement (disp) bits give a signed range of -128 to 127 locations to be jumped if the condition is true.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JAZ

Condition: Jump if (A) == 0 (jump if A is zero)

Opcode:	0131000 (octal) = 0xB200
Format:	JAZ <displacement>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	jump_on_condition

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								displacement							

Description

If condition true, jump to the address of the program counter plus the value of displacement.
If condition false, continue program execution at (P) + 1.

'displacement' = The eight displacement (disp) bits give a signed range of -128 to 127 locations to be jumped if the condition is true.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JMP

Jump - Unconditional jump to specified address
The next instruction is taken from the effective address of the JMP instruction(the effective address is loaded into the program counter).

Opcode:

0124000 (octal) = 0xA800

Format:

JMP <address mode> <displacement>

Privilege:

User

Class:

memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JNC

Increment X and jump if X is negative

Opcode:	0132400 (octal) = 0xB500
Format:	JNC <displacement>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	jump_on_condition

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								displacement							

Description

$X = X + 1$
Condition: Jump if $(X) < 0$ (jump if X is negative)

If condition true, jump to the address of the program counter plus the value of displacement.
If condition false, continue program execution at $(P) + 1$.

'displacement' = The eight displacement (disp) bits give a signed range of -128 to 127 locations to be jumped if the condition is true.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JPC

Increment X and jump if X is positive

Opcode:	0132000 (octal) = 0xB400
Format:	JPC <displacement>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	jump_on_condition

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								displacement							

Description

$X = X + 1$
Condition: Jump if (X) > 0 (jump if X is positive)

If condition true, jump to the address of the program counter plus the value of displacement.
If condition false, continue program execution at (P) + 1.

'displacement' = The eight displacement (disp) bits give a signed range of -128 to 127 locations to be jumped if the condition is true.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JPL

Jump if Plus - Jump to specified address if the result of the last operation was positive (sign bit is 0)

The contents of the program counter are transferred to the L register and the next instruction is taken from the effective address of the JPL instruction.

Note that the L register points to the instruction after the jump(the program counter incremented before transfer to the L register).

Opcode: 134000 (octal) = 0xB800
Format: JPL <address mode> <displacement>
Privilege: User
Class: memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JXN

Condition: Jump if (X) < 0 (jump if X negative)

Opcode:	0133400 (octal) = 0xB700
Format:	JXN <displacement>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	jump_on_condition

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								displacement							

Description

If condition true, jump to the address of the program counter plus the value of displacement.
If condition false, continue program execution at (P) + 1.

'displacement' = The eight displacement (disp) bits give a signed range of -128 to 127 locations to be jumped if the condition is true.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

JXZ

Condition: Jump if (X) == 0 (jump if X is zero)

Opcode:	0133000 (octal) = 0xB600
Format:	JXZ <displacement>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	jump_on_condition

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								displacement							

Description

If condition true, jump to the address of the program counter plus the value of displacement.
If condition false, continue program execution at (P) + 1.

'displacement' = The eight displacement (disp) bits give a signed range of -128 to 127 locations to be jumped if the condition is true.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

LACB

Load the A register from the core map-table bank (CMBNK).

(A) <--- (ea)

ea = (B) + Δ = CMBNK entry

Δ: 3-bit displacement added to B included in the instruction opcode (bits 5-3)

Opcode:	140702 (octal) = 0xC1C2
Format:	LACB <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

LASB

Load the A register with the contents of the segment-table bank (STBNK).

(A) <--- (ea)

ea = (B) + Δ = STBNK entry

Δ: 3-bit displacement added to B included in the instruction opcode (bits 5-3)

Opcode:	140700 (octal) = 0xC1C0
Format:	LASB <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

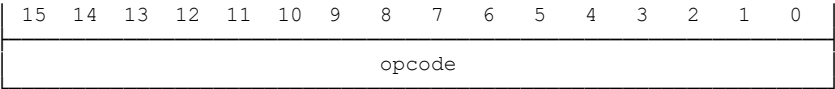
LBIT

Load single bit accumulator (K) with logical memory bit.

(X) points to the start of a bit array
(A) points to the bit within the array

Opcode:	140510 (octal) = 0xC148
Format:	LBIT
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

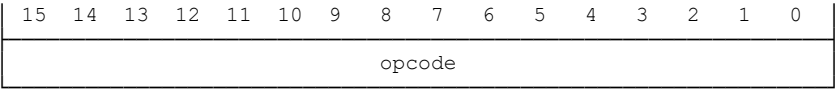
LBITP

Load single bit accumulator (K) with physical memory bit.

- (T) points to the bank number containing the bit array
- (X) points to the start of a bit array
- (A) points to the bit within the array

Opcode:	140511 (octal) = 0xC149
Format:	LBITP
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

LBYT

Load byte from memory to A register

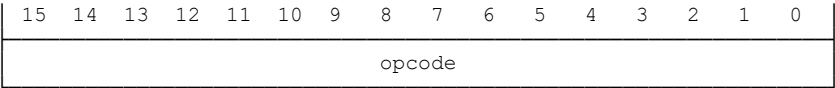
- Addressing: $EL = (T) + (X)/2$
- If least significant bit of X = 1: Load right byte
 - If least significant bit of X = 0: Load left byte

Load the byte addressed by the contents of the T and X register into the lower byte of the A register. The higher byte of the A register is cleared.

The contents of T point to the beginning of a character string and the contents of X to a byte within the string.

Opcode:	142200 (octal) = 0xC480
Format:	LBYT
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

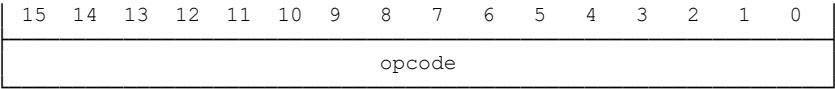
LBYTP

Load the A register with a byte from physical memory.

- (D) points to the bank number containing the byte array
- (T) points to the start of a byte array
- (X) points to the actual byte within the array

Opcode:	140514 (octal) = 0xC14C
Format:	LBYTP
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

LDA

Load A register

Load the contents of the memory location pointed to by the effective address into the A register.

Opcode:	044000 (octal) = 0x4800
Format:	LDA <addressing_mode> <disp>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

LDATX

Load A register

$(A) \leftarrow (ea)$

Load the contents of the physical memory location pointed to by the effective address into the A register.

Opcode:	143300 (octal) = 0xC6C0
Format:	LDATX <disp>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

LDBTX

Load B register

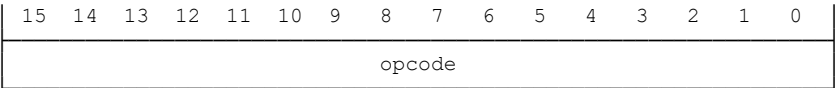
$(B) \leftarrow 177000_8 \text{ OR } (2(ea))$

Load the contents of the physical memory location pointed to by twice the effective address contents into the B register, then OR the value with 177000₈.

See description for usage.

Opcode:	143303 (octal) = 0xC6C3
Format:	LDBTX <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format



LDD

Load double word

$A \leftarrow (ea)$
 $D \leftarrow (ea) + 1$

Load the contents of the memory location pointed to by the effective address into the A register
Load the contents of the memory location pointed to by the effective address plus one into the D register.

Opcode:	024000 (octal) = 0x2800
Format:	LDD <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15 14 13 12 11	10 9 8	7 6 5 4 3 2 1 0
	X I B	
opcode	addressing_mode	displacement

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

LDDTX

Load double word

- (A) ← (ea)
- (D) ← (ea + 1)

Load the contents of the physical memory location pointed to by the effective address into the A register and the contents of the effective address plus one into the D register.

Opcode:

143302 (octal) = 0xC6C2

Format:

LDDTX <displacement>

Mask:

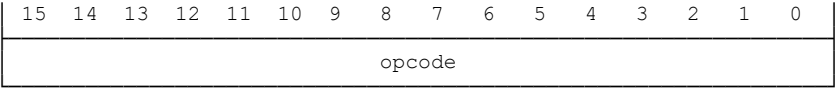
1111_1111_1100_0111

Privilege:

Privileged

Class:

Bit Format



LDF

Load floating accumulator (TAD) from memory (FW)

Memory format:

(T) ← EL

(A) ← EL+1

(D) ← EL+2

Load the contents of the memory location pointed to by the effective address into the T register, the contents of the effective address plus one into the A register and the contents of the effective address plus two into the D register.

Opcode:	034000 (octal) = 0x3800
Format:	LDF <addressing_mode> <disp>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

LDT

Load T register

$(T) \leftarrow (ea)$

Load the contents of the memory location pointed to by the effective address into the T register.

Opcode:	050000 (octal) = 0x5000
Format:	LDT <addressing_mode> <disp>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

LDX

Load X register

$(X) \leftarrow (ea)$

Load the contents of the memory location pointed to by the effective address into the X register.

Opcode:	054000 (octal) = 0x5800
Format:	LDX <addressing_mode> <disp>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

LDXTX

Load X register

$(X) \leftarrow (ea)$

Load the contents of the physical memory location pointed to by the effective address into the X register.

Opcode:	143301 (octal) = 0xC6C1
Format:	LDXTX <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

LEAVE

Leave stack.

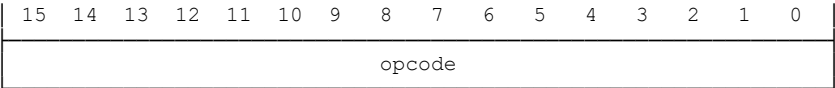
This instruction saves the previous stack pointer in LINK.
The B register is restored to its previous value (PREVB) and the stack is left by loading the P register (program counter) with the return address (LINK).

(P) ← (B = 200₈) {LINK}
(B) ← (B = 177₈) {PREVB}

Format:
LEAVE

Opcode:	0140136 (octal) = 0xC05E
Format:	LEAVE
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

LRB

Load register block

Load the contents of a memory block pointed to by the X register into the register block of the program level given in the instruction.

If the instruction specifies the current program level, the P register (program counter) is not loaded from memory and is unchanged.

Opcode:	152600 (octal) = 0xD580
Format:	LRB <level>
Mask:	1111_1111_1100_0000
Privilege:	Privileged
Class:	register_block

Bit Format

15	14	13	12	11	6	5	4	3	2	1	0
opcode					level				type		

Description

The memory block pointed to by the X register is loaded into the register block for the specified program level. The mapping is as follows:

- P (level N) ← (ea)
- X (level N) ← (ea + 1)
- T (level N) ← (ea + 2)
- A (level N) ← (ea + 3)
- D (level N) ← (ea + 4)
- L (level N) ← (ea + 5)
- STS (level N) ← (ea + 6)
- B (level N) ← (ea + 7)

Where (ea) is the address in memory pointed to by the X register.

If the current program level is specified, the P register is not loaded and remains unchanged.

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
level	6-3	uint4	The level to load the register block to
type	2-0	enum	The type of register block function

Examples

LRB 160₈ (152760₈)

Load the memory block pointed to by the X register into the register on program level 14.

- P (level 14) ← (ea)
- X (level 14) ← (ea + 1)
- ↓ ↓
- B (level 14) ← (ea + 7)

LRB 160

LWCS

Writable Control Store Instruction

This instruction is PRIVILEGED and only available to:

- programs running in system mode (rings 2-3)
- programs running without memory protection

LWCS is a no-operation in the ND-110.

The ND-110 is software compatible but not microcode compatible and writing to the writable control store has no meaning in the ND-110.

A no-operation is executed so that programs written for the ND-100 and NORD-10 can continue.

Unused areas of the microprogram can be read or written to using the TRR CS or TRA CS instruction.

Further information on the LWCS instruction for the ND-100 can be found in the ND-100 Reference Manual (ND-06.014).

Opcode: 143500 (octal) = 0xC740
Format: LWCS
Mask: 1111_1111_1111_1111
Privilege: Privileged
Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

LXCB

Load the X register from the core table bank (CMBNK).

(X) <--- (ea)

ea = (B) + Δ = CMBNK entry

Δ: 3-bit displacement added to B included in the instruction opcode (bits 5-3)

Opcode:	140705 (octal) = 0xC1C5
Format:	LXCB <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

LXSB

Load the X register from the segment table bank (STBNK).

(X) <--- (ea)

ea = (B) + Δ = STBNK entry

Δ: 3-bit displacement added to B included in the instruction opcode (bits 5-3)

Opcode:	140704 (octal) = 0xC1C4
Format:	LXSB <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

MCL

Masked clear

The A register is used as a mask to clear bits within the selected internal register. Setting a bit in the A register clears the corresponding bit in the internal register.

See table for internal registers that allow MCL.

Opcode:	0150200 (octal) = 0xD080
Format:	MCL <internal_register>
Mask:	1111_1111_1111_0000
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode												internal_register			

Operands

Name	Bits	Type	Description
opcode	15-4	opcode	The opcode determines what type of operation occurs
internal_register	3-0	enum	The CPU internal register

Examples

MCL STS (150201₈) [(A) = 000100₈]

Clear the carry flag (bit 6) in the status (STS) register.

```
MCL STS
```

MIN

Memory increment and skip next instruction if zero (EA): = (EA) + 1

$(ea) \leftarrow (ea) + 1$

$(P) \leftarrow (P) + 2$ IF new $(ea) = 0$

The contents of the memory location pointed to by the effective address are incremented by one. If the new memory location when incremented becomes zero, the next instruction is skipped.

Opcode: 040000 (octal) = 0x4000
Format: MIN <addressing_mode> <displacement>
Mask: 1111_1000_0000_0000
Privilege: User
Class: memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

MIX3

Multiply index by 3.

$(X) \leftarrow [(A)-1] * 3$

Take the contents of the A register as an operand and subtract one. Multiply the result by three and place it in the X register.

Opcode:

143200 (octal) = 0xC680

Format:

MIX3

Mask:

1111_1111_1111_1111

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

MON

The MON instruction is used in special different contexts when running under an operating system. It provides system call functionality through different monitor call numbers.

Opcode:

153000 (octal) = 0xD600

Format:

MON <monitor_call_number>

Mask:

1111_1111_0000_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								monitor_call_number							

Description

The unsigned <monitor_call_number> allows 256 monitor calls. This field is loaded into the T register on level 14. The higher byte of the T register is sign extended.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
monitor_call_number	7-0	numeric	Monitor call number that determines the system function

Examples

End of program or stop

```
MON 0
```

Read a character from specified device into the A register

```
MON 1
```

Output the character contained in the A register on the specified device

```
MON 2
```

MOVB

Move byte.

This instruction moves a block of bytes from the memory location addressed by the source operand to that of the memory location addressed by the destination operand.

After execution, bit 15 of the D and T registers point to the end of the field that has been moved. The field length of the D register (source) equals zero and the T register (destination) field length is equal to the number of bytes moved.

Opcode:	0140131 (octal) = 0xC059
Format:	MOVB
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

MOVBF

Move bytes forward.

This instruction moves a block of bytes from the memory location addressed by the source operand to that of the memory location addressed by the destination operand.

After execution, bit 15 of the D and T registers point to the end of the field that has been moved. The field length of the D register (source) equals zero and the T register (destination) field length is equal to the number of bytes moved.

Opcode:	0140132 (octal) = 0xC05A
Format:	MOVBF
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

MOVEW

Word block move.

This instruction moves a block of words from one area of memory to another. The type of transfer is determined by the opcode field $\Delta\Delta$, with the base octal code 1431 $\Delta\Delta$.

<ascii-table>

$\Delta\Delta_8$	move from:	move to:
00 ₈	PT	PT
01 ₈	PT	APT
02 ₈	PT	phy.memory*
03 ₈	APT	PT
04 ₈	APT	APT
05 ₈	APT	phy.memory*
06 ₈	phy.memory	PT*
07 ₈	phy.memory	APT*
10 ₈	phy.memory	phy.memory*

</ascii-table>

* = privileged instruction

PT = normal page table

APT = alternative page table

phy.memory = physical memory

The following registers control the transfer:

- A and D: source address
- X and T: destination address
- L: number of words to be moved (maximum 2048 words)

For physical memory-block moves, A and/or X are incremented when D and/or T overflow.

If the L register contains a value greater than 2048 ($L = 4000_8$), no words are moved and A, D, T, and X remain unchanged.

After transfer, the registers contain:

- A, D, T, X: addresses after the last moved word
- L: zero

Special cases:

If the memory management system is off (bit PTM of the STS register is zero), bank 0 of physical memory is addressed and the following transfer fields become equivalent:

- $\Delta\Delta = 00 = 01 = 03 = 04$
- $\Delta\Delta = 02 = 05$
- $\Delta\Delta = 06 = 07$

MOVEW can be interrupted. L, A, D, X, T, and P registers are then changed to restart execution.

Opcode: 0143100 (octal) = 0xC640

Format:

Mask:

Privilege:

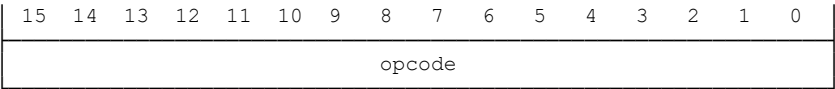
Class:

MOVEW

1111_1111_1111_1000

User

Bit Format



MPY

Multiply integer
 $A \leftarrow A * (EL)$

Multiply the contents of the memory location pointed to by the effective address with the contents of the A register, leaving the result in A.

Opcode:	120000 (octal) = 0xA000
Format:	MPY <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Description

****Flags affected****
 $O \leftarrow 1; Q \leftarrow 1$
If an overflow occurs, that is if the result has an absolute value greater than decimal 32767

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

Flags: O, Q

MST

Masked set

The A register is used as a mask to set bits within the selected internal register. Setting a bit in the A register sets the corresponding bit in the internal register.

See table for internal registers that allow MST.

Opcode:	0150300 (octal) = 0xD0C0
Format:	MST <register>
Mask:	1111_1111_1111_0000
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode												internal_register			

Operands

Name	Bits	Type	Description
opcode	15-4	opcode	The opcode determines what type of operation occurs
internal_register	3-0	enum	The CPU internal register

NLZ

Normalize

Convert the number in A to a floating number in TAD

Opcode:

Format:

Mask:

Privilege:

Class:

0151400 (octal) = 0xD300

NLZ <scaling_factor>

1111_1111_0000_0000

User

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								scaling_factor							

Description

The number in the A register is converted to its floating point equivalent in the floating point accumulator (A and D registers), using the scaling factor given, For integers, a scaling factor of +16 will give a floating point number with the same value as the integer.
The larger the scaling factor, the larger the Floating point number.

The D register will be cleared when using single precision fixed point numbers.

Comments on 48-bit floating point CP
For the ND-110, 48-bit floating point CPU option, a further register (T) and memory location fea + 2) are used. In this case, the T register is linked to location ea, A to ea + 1 and D to ea + 2.

How to test for a 32-bit or 48-bit floating point CPU:
SAT 0
SAA 1
NLZ 20

This tests whether T is changed, if so, the CPU is 48-bit; otherwise it is 32-bit.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode for floating conversion
scaling_factor	7-0	numeric	Scaling factor in range -128 to 127 (gives converting range from 10^-39 to 10^39)

Examples

Convert the number in A to a floating number in TAD

NLZ

Integer to floating conversation

NLZ +20

OPCOM

Operator Communication

This instruction is PRIVILEGED and only available to:

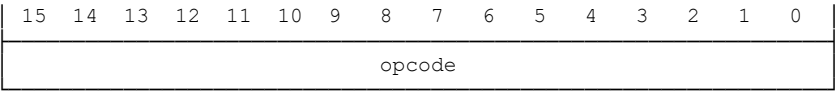
- programs running in system mode (rings 2-3)
- programs running without memory protection

This instruction allows the programmer to use a terminal in direct communication with the CPU board. When the CPU is running, MOPC can be used to read input from the console. This is the software equivalent to pressing the OPCOM button on the control panel of the ND-110.

Opcode:
Format:
Mask:
Privilege:
Class:

150400 (octal) = 0xD100
OPCOM
1111_1111_1111_1111
Privileged

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

ORA

Logical inclusive OR to A register
 $A \leftarrow A \mid (EA)$

Perform a bitwise OR operation between the contents of the A register and the contents of the memory location pointed to by the effective address, leaving the result in A.

Opcode:	074000 (octal) = 0x7800
Format:	ORA <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

PACK

Convert to BCD.

(op2) ← (op1) in BCD format

Convert the first operand from its ASCII format to BCD format, placing the result in the second operand location.

****Conversion process:****

PACK carries out the following steps:

- 1. Checks the sign and digits of the operand (op1) are encoded as ASCII digits. (Reporting illegal codes as error code 2.)
- 2. Takes the 4 least significant bits of each ASCII digit as the equivalent BCD digit.
- 3. Converts the ASCII sign of the operand to BCD:

ASCII BCD Sign
53₈ 14₈ +
55₈ 15₈ -

- Note: If bit 13 of the descriptor D2 is set, then the code 17₈ (BCD unsigned) is used.
- 4. Extends the second operand field (op2) with zeros if the result is too small to fill the field. Reports overflow has occurred (error code 3) if the second operand field length is too short to contain the significant digits of the result (the remaining digits are ignored).

****Error code:****

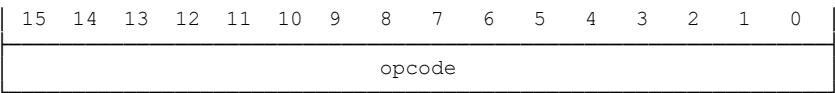
An error code is placed in bits 0 to 4 of the D register if an illegal code conversion is attempted or if overflow occurs. The first detected error will be reported.

Error code: 2 illegal code†
 3 overflow

† bit 15 of both the A and D registers point to the byte containing the illegal code.

Opcode:	0140124 (octal) = 0xC054
Format:	PACK
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

PIOF

Memory management and interrupt system OFF

Disables both the memory management and interrupt systems. This combines the functions of the IOF and POF instructions.

Before_use:
Check conditions of the IOF instruction.

Opcode:	150405 (octal) = 0xD105
Format:	PIOF
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

PION

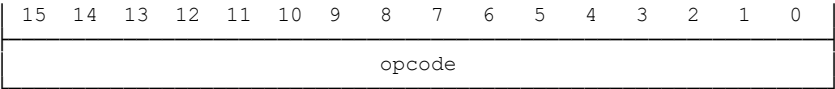
Memory management and interrupt system ON

Enable both the memory management and interrupt systems. This combines the functions of the ION and PON instructions.

Opcode:
Format:
Mask:
Privilege:
Class:

150412 (octal) = 0xD10A
PION
1111_1111_1111_1111
User

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

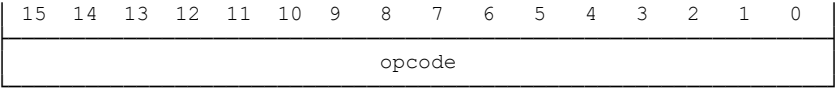
POF

Memory management OFF

Disable memory management system. The next instruction will be taken from a physical address given by the address following the POF instruction.
Note: The CPU will be in an unrestricted mode without any hardware protection features - all instructions are legal and all memory accessible.

Opcode:	150404 (octal) = 0xD104
Format:	POF
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

PON

Memory management ON

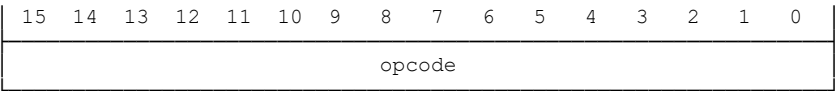
Enable memory management system. The next instruction after PON will then use the pageindex table specified by PCR.

BEFORE USE ENSURE:

- Interrupt system is enabled,
- Internal hardware interrupts are enabled,
- Page tables and PCR registers are initialized.

Opcode:	150410 (octal) = 0xD108
Format:	PON
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

RADD

Add source to destination (dr): = (dr) + (sr)

Opcode: 0146000 (octal) = 0xCC00
Format: RADD [sub-instruction(s)] <source> <destination>
Mask: 1111_1100_0000_0000
Privilege: User
Class: register_arithmetic

Bit Format

15	14	13	12	11	10	9	8	8	7	7	6	5	4	3	2	1	0
opcode					rad	ADC	AD1	CM2	CM1	CLD	source				destination		

Description

These instructions specify operations between source (sr) and destination (dr) registers.

Optional sub-instructions:

- ****CLD****: CLD=1. Zero is used instead of the destination register as an operand (dr register contents are unchanged).
- ****CM1****: CM1=1. The 1's complement of the source register is used as an operand (sr register contents are unchanged).
- ****ADC****: This mnemonic represents C=0 I=1. Add previous carry to destination register.
- ****AD1****: This mnemonic represents C=1 I=0. Add 1 to destination register.
- ****CM2****: This compound sub-instruction is equivalent to CM1 ADC.

Flags affected:

RADD instructions affect the carry (C) and overflow (O and Q) flags as follows:

- ****C = 1****: If a carry occurs from the signed bit positions of the adder.
- ****O = 1 ; Q = 1****: If an overflow occurs, that is if the signs of the two operands are equal and the sign of the result is different.
- ****O = 1 ; Q = 0****: If overflow does not exist, the dynamic overflow flag (O) is reset while the static overflow flag (Q) is left unchanged.

Instruction combinations

COPY

RADD <sr> <dr> dr ← dr + sr
 RADD CLD <sr> <dr> dr ← sr ≡ COPY
 RADD CM1 <sr> <dr> dr ← dr + ~sr
 RADD CM1 CLD <sr> <dr> dr ← ~sr
 RADD AD1 <sr> <dr> dr ← dr + sr + 1
 RADD CLD AD1 <sr> <dr> dr ← sr + 1
 RADD CM1 AD1 <sr> <dr> dr ← dr - sr ≡ RSUB
 RADD CM1 CLD AD1 <sr> <dr> dr ← -sr
 RADD ADC <sr> <dr> dr ← dr + sr + c
 RADD CLD ADC <sr> <dr> dr ← sr + c
 RADD CM1 ADC <sr> <dr> dr ← dr + ~sr + c
 RADD CM1 CLD ADC <sr> <dr> dr ← ~sr + c

RINC

RADD AD1 <dr> is equivalent to RINC <dr>, increment dr register by one.

RDCR

RADD CM1 <dr> is equivalent to RDCR <dr>, decrement dr register by one.

RCLR

RADD CLD 0 (COPY 0) is equivalent to RCLR <dr>, register clear.

EXIT

RADD CM1 SL DP (COPY SL DP) is equivalent to EXIT, return from subroutine.

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode for register inclusive OR
rad	10	constant	Arithmetic operation flag
ADC	9	flag	Also add old carry to destination
AD1	8	flag	Also add one to destination
CM1	7	flag	Use one's complement of source
CM2	8-7	flag	Two's complement (CM1 AD1)
CLD	6	flag	Clear destination before operation
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

Add the contents of the B register to the contents of the A register, leaving the result in the A register.

```
RADD SB DA
```

Add the contents of the A and X registers together and place the result in the X register. (146057₈)

```
RADD SA DX
```

Use zero as the destination operand, add the X register contents and leave the result in B. THAT IS copy the contents of the X register into B. (146173₈)

Equivalent to COPY SX DB.

```
RADD CLD SX DB
```

Copy the negative value of the X register contents into B. (146773₈)

Equivalent to COPY CLD AD1 or RSUB CM1.

```
RADD CM1 CLD AD1 SX DB
```

RAND

Logical AND to destination
(dr): = (dr) & (sr)

Opcode:	144400 (octal) = 0xC900
Format:	RAND [sub-instruction(s)] <source> <destination>
Mask:	1111_1111_1100_0000
Privilege:	User
Class:	register_logical

Bit Format

15	14	13	12	11	10	9	8	8	7	7	6	5	4	3	2	1	0
opcode					rad	ADC	AD1	CM2		CM1	CLD	source			destination		

Description

Optional sub-instructions:

- **CLD=1**: Zero is used instead of the destination register as an operand (dr register contents are unchanged).
- **CM1=1**: The 1's complement of the source register is used as an operand (sr register contents are unchanged).

Instruction combinations

RAND <sr> <dr> dr ← dr AND sr
RAND CLD <sr> <dr> dr ← 0
RAND CM1 <sr> <dr> dr ← dr AND ~sr
RAND CM1 CLD <sr> <dr> dr ← 0

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode for register inclusive OR
rad	10	constant	Logical operation flag
ADC	9	flag	Also addd old carry to destination
AD1	8	flag	Also add one to destination
CM1	7	flag	Use one's complement of source
CM2	8-7	flag	Two's complement (CM1 AD1)
CLD	6	flag	Clear destination before operation
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Examples

AND the contents of the L and X registers. Store the result in the X register. (144447₈)

```
RAND SL DX
```

AND the contents of the T and B registers, taking the 1's complement of the sr as the source operand. (144663₈)
Result in B.

```
RAND CM1 ST DB
```

RCLR

Register clear

Opcode:	0146100 (octal) = 0xCC40
Format:	RCLR <destination>
Mask:	1111_1111_1111_1000
Privilege:	User
Class:	register_dst

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	2	1	0
opcode												destination		

Description

RCLR is a compound mnemonic for "COPY 0" (or RADD CLD 0).

Operands

Name	Bits	Type	Description
opcode	15-4	opcode	The opcode for register arithmetic
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

Clear the contents of the A register.

```
RCLR DA
```

RDCR

Register decrement

$dr \leftarrow dr - 1$

RDCR is a compound mnemonic for RADD CM1.

Opcode:	146200 (octal) = 0xCC80
Format:	RDCR <destination>
Mask:	1111_1111_1111_1000
Privilege:	User
Class:	register_dst

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	2	1	0
opcode												destination		

Operands

Name	Bits	Type	Description
opcode	15-4	opcode	The opcode for register arithmetic
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

Decrement the contents of the B register by one. (146203₈)

```
RDCR DB
```

RDIV

Divide double accumulator with source; quotient in A, remainder in D

$A = AD / (sr)$
 $(AD = A * (sr) + D)$

Opcode:	141600 (octal) = 0xC380
Format:	RDIV <source> <destination>
Mask:	1111_1111_1100_0000
Privilege:	User
Class:	register_src_dst

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode										source			destination		

Description

The 32-bit signed integer held in the double accumulator AD is divided by the contents of sr. If division causes overflow, the error flag (Z) is set.

The numbers are fixed point integers with the fixed point after the rightmost position.

Operands

Name	Bits	Type	Description
opcode	15-6	opcode	The opcode for register arithmetic
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Flags: Z, C, O, Q

RDUS

Read a word without using cache

- (T) points to the virtual memory word to be accessed
- (A) ← memory word addressed by T

The address given by the T register is a logical memory address. It is normally translated into a physical address using page tables (if the memory management system is on). The contents of the location addressed by T are loaded into the A register. The old content of the memory address is always read from the memory and never from cache.

The execution time of this instruction includes two read bus cycles (semaphore cycles – see ND-110 Functional Description Manual ND.06.027).

Opcode:	140127 (octal) = 0xC057
Format:	RDUS
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

RDUSP

Read a physical memory word without using cache.

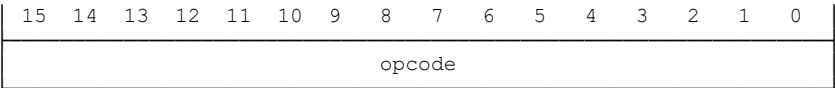
- (T) points to the physical memory bank to be accessed
- (X) points to the address within the bank
- (A) is loaded with the memory word

The old content of the memory address is always read from the memory and never from cache.

Note: The execution time of this instruction includes two read-bus cycles (The CPU uses semaphore cycles - see ND-110 Functional Description Manual ND.06.027)

Opcode:	140517 (octal) = 0xC14F
Format:	RDUSP
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

REMP

Remove page from page list.

Opcode:	140503 (octal) = 0xC143
Format:	REMP
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

REPT

Enter reentrant segment in page tables. (See Appendix B for a software description.)

Opcode:

140507 (octal) = 0xC147

Format:

REPT

Mask:

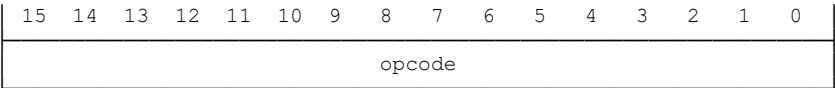
1111_1111_1111_1111

Privilege:

Privileged

Class:

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

REX

Reset extended address mode

Opcode:	0150407 (octal) = 0xD107
Format:	REX
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

REXO

Logical exclusive OR

$(dr) := (dr) \oplus (sr)$

Opcode:	0145000 (octal) = 0xCA00
Format:	REXO [flags] <source> <destination>
Mask:	1111_1111_1100_0000
Privilege:	User
Class:	register_logical

Bit Format

15	14	13	12	11	10	9	8	8	7	7	6	5	4	3	2	1	0
opcode					rad	ADC	AD1	CM2	CM1	CLD	source			destination			

Description

Optional sub instructions:
CLD=1 Zero is used instead of the destination register as an operand (dr register contents are unchanged).
CM1=1 The 1's complement of the source register is used as an operand (sr register contents are unchanged).

Instruction combinations:

REXO <sr> <dr> dr ← dr XOR sr
REXO CLD <sr> <dr> dr ← sr
REXO CML <sp> <dr> dr ← dr XOR sr's (1's complement)
REXO CML CLD <sr> <dr> dr ← sr (1's complement)

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode for register inclusive OR
rad	10	constant	Logical operation flag
ADC	9	flag	Also addd old carry to destination
AD1	8	flag	Also add one to destination
CM1	7	flag	Use one's complement of source
CM2	8-7	flag	Two's complement (CM1 AD1)
CLD	6	flag	Clear destination before operation
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Examples

Exclusive OR the contents of the B and Y registers, leaving the result in B

```
REXO ST DB
```

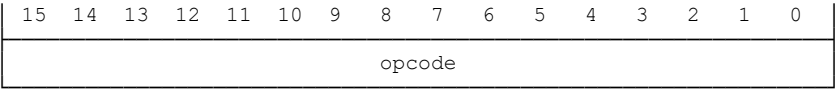
RGLOB

Examine global pointers.

- (T) <--- bank number of segment table (STBNK)
- (A) <--- start address within bank (STSRT)
- (D) <--- bank number of core map table (CMBNK)

Opcode:	140501 (octal) = 0xC141
Format:	RGLOB
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

RINC

Register increment

$dr \leftarrow dr + 1$

RINC is a compound mnemonic for RADD AD1.

Opcode:	146400 (octal) = 0xCD00
Format:	RINC <destination>
Mask:	1111_1111_1111_1000
Privilege:	User
Class:	register_dst

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	2	1	0
opcode												destination		

Operands

Name	Bits	Type	Description
opcode	15-4	opcode	The opcode for register arithmetic
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

Increment the contents of the A register by one.

```
RINC DA
```

RMPY

Multiply source with destination; result in double accumulator
 $AD \leftarrow (sr) * (dr)$

Opcode:	141200 (octal) = 0xC280
Format:	RMPY <source> <destination>
Mask:	1111_1111_1100_0000
Privilege:	User
Class:	register_src_dst

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode										source			destination		

Description

The sr and dr registers hold the two operands to be multiplied together.
The result is a 32-bit signed integer held in the A and D register (the A register contains the 16 most significant bits).

Operands

Name	Bits	Type	Description
opcode	15-6	opcode	The opcode for register arithmetic
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

Multiply the contents of the A and X registers together, leaving the result in the A and D registers.

```
RMPY SA DX
```

RORA

OR register

$dr \leftarrow dr \text{ OR } sr$

Opcode:	145400 (octal) = 0xCB00
Format:	RORA <sub-instruction(s)> <source> <destination>
Mask:	1111_1111_1100_0000
Privilege:	User
Class:	register_logical

Bit Format

15	14	13	12	11	10	9	8	8	7	7	6	5	4	3	2	1	0
opcode					rad	ADC	AD1	CM2	CM1	CLD	source			destination			

Description

Optional sub-instructions:

- **CLD=1****:** Zero is used instead of the destination register as an operand (dr register contents are unchanged).
- **CM1=1****:** The 1's complement of the source register is used as an operand (sr register contents are unchanged).

Instruction combinations

RORA <sr> <dr> $dr \leftarrow dr \text{ OR } sr$
RORA CLD <sr> <dr> $dr \leftarrow sr$
RORA CM1 <sr> <dr> $dr \leftarrow dr \text{ OR } \sim sr$
RORA CM1 CLD <sr> <dr> $dr \leftarrow \sim sr$

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode for register inclusive OR
rad	10	constant	Logical operation flag
ADC	9	flag	Also addd old carry to destination
AD1	8	flag	Also add one to destination
CM1	7	flag	Use one's complement of source
CM2	8-7	flag	Two's complement (CM1 AD1)
CLD	6	flag	Clear destination before operation
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

OR the contents of the B and T registers leaving the result in B. (145463₈)

```
RORA ST DB
```

RSUB

Register subtract

$dr \leftarrow dr - sr$

RSUB is a compound mnemonic for RADD AD1 CM1 (or RADD CM2 using the AD1 CM2 compound mnemonic).

Opcode:	146600 (octal) = 0xCD80
Format:	RSUB <sub-instruction> <source> <destination>
Mask:	1111_1111_1100_0000
Privilege:	User
Class:	register_arithmetic

Bit Format

15	14	13	12	11	10	9	8	8	7	7	6	5	4	3	2	1	0
opcode					rad	ADC	AD1	CM2		CM1	CLD	source			destination		

Description

The following sub-instruction is allowed:

- **CLD**

Note:
Sometimes the RADD form will be more readable.

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode for register inclusive OR
rad	10	constant	Arithmetic operation flag
ADC	9	flag	Also addd old carry to destination
AD1	8	flag	Also add one to destination
CM1	7	flag	Use one's complement of source
CM2	8-7	flag	Two's complement (CM1 AD1)
CLD	6	flag	Clear destination before operation
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

Subtract the contents of the T register from the contents of the B register leaving the result in B.

```
RSUB ST DB
```

SAA

Set argument to A

Opcode:

170400 (octal) = 0xF100

Format:

SAA <number>

Mask:

1111_1111_0000_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								number							

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
number	7-0	numeric	8-bit argument extended to 16 bits using sign extension

Flags: C, O

Examples

Set the A register to 10.

```
SAA 10
```

SAB

Set argument to B

Opcode:	170000 (octal) = 0xF000
Format:	SAB <number>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								number							

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
number	7-0	numeric	8-bit argument extended to 16 bits using sign extension

Flags: C, O

Examples

The contents of the B register are set to 177752, bits 0-7 have been extended with ones as the argument is negative; bits 0-7 have the argument in its 2's complement form.

```
SAB -26
```

SACB

Store the A register in the core map table bank (CMBNK).

(ea) <--- (A)

ea = (B) + Δ = CMBNK entry

Δ: 3-bit displacement added to B included in the instruction opcode (bits 5-3)

Opcode:	140703 (octal) = 0xC1C3
Format:	SACB <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

SAD

Shift A and D registers connected
Every shift instruction places the last bit discarded in the multi-shift flag (M). M can be used as an input for the next shift instruction. M is bit 8 of the STS Register.

Opcode:

0154600 (octal) = 0xD980

Format:

SAD <shift_type> <shift_counter>

Mask:

1111_1001_1000_0000

Privilege:

User

Class:

shift_instructions

Bit Format

15	14	13	12	11	10	9		6	5	4	3	2	1	0	0
opcode					shift_type		register	mask6	shift_counter					SHR	

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
shift_type	10-9	enum	
register	7-8	enum	The register to shift
mask6	6	numeric	Always 0
SHR	0	assembler_flag	This a feature of the assembler. This mnemonic can be used to specify shift right, so that instead of calculating the 2's complement for the number of right shifts required, SHR can be used.
shift_counter	5-0	int6	The number of bits to shift the register. Bits 0-4 are the number of shifts. Bit 5=1, then shift right (max 32 times) Bit 5=0, then shift left (max 31 times)

Flags: M

SASB

Store the A register contents in the segment table bank (STBNK).

(ea) <--- (A)

ea = (B) + Δ = STBNK entry

Δ: 3-bit displacement added to B included in the instruction opcode (bits 5-3)

Opcode:	140701 (octal) = 0xC1C1
Format:	SASB <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

SAT

Set argument to T

Opcode:

171000 (octal) = 0xF200

Format:

SAT <number>

Mask:

1111_1111_0000_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								number							

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
number	7-0	numeric	8-bit argument extended to 16 bits using sign extension

Flags: C, O

Examples

Set the T register to 20₈.

SAT 20₈

SAX

Set argument to X

Opcode:	171400 (octal) = 0xF300
Format:	SAX <number>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								number							

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
number	7-0	numeric	8-bit argument extended to 16 bits using sign extension

Flags: C, O

Examples

Set the X register to -5.

SAX -5

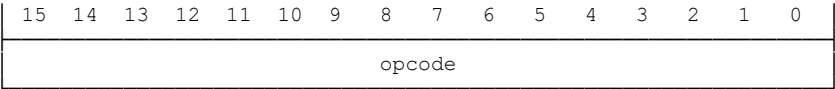
SBIT

Store the single bit accumulator (K) in a logical memory bit.

- (X) points to the start of a bit array
- (A) points to the bit within the array

Opcode:	140512 (octal) = 0xC14A
Format:	SBIT
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

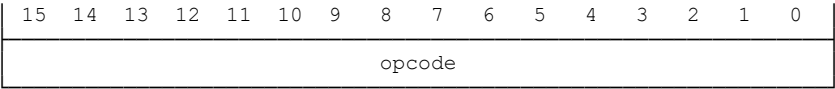
SBITP

Store the single bit accumulator (K) in a physical memory bit.

- (T) points to the bank number containing the bit array
- (X) points to the start of a bit array
- (A) points to the bit within the array

Opcode:	140513 (octal) = 0xC14B
Format:	SBITP
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

SBYT

Store byte from A register to memory
Addressing: $EL = (T) + (X)/2$
- If least significant bit of X = 1: Store right byte
- If least significant bit of X = 0: Store left byte

The contents of T point to the beginning of a character string and the contents of X to a byte within the string.

Opcode:

142600 (octal) = 0xC580

Format:

SBYT

Mask:

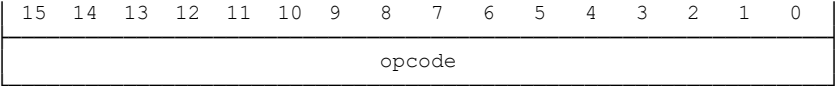
1111_1111_1111_1111

Privilege:

User

Class:

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

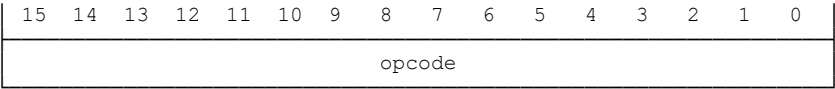
SBYTP

Store a byte in physical memory.

- (D) points to the bank number containing the byte array
- (T) points to the start of a byte array
- (X) points to the actual byte within the array

Opcode:	140515 (octal) = 0xC14D
Format:	SBYTP
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

SETPT

Set page tables.

Opcode:	140300 (octal) = 0xC0C0
Format:	SETPT
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Description

This instruction can replace the following instructions:

```
SETPT: JXZ *78
      LDDTX 208
      BSET ZRO 1308 DA
      LDBTX 108
      STD ,B
      LDXTX 00
      JMP *-68
```

Each time the loop is executed (until X becomes zero) two consecutive physical memory locations addressed by X are loaded into the A and D registers.

The word in A is the protect field of the page table, bit 11 (the PGU bit) is cleared to set the page table. The double word (in A and D) is then stored in two consecutive locations pointed to by the contents of the B register, the page table address.

* is the mnemonic for P relative addressing.

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

SEX

Set extended address mode

Opcode:	150406 (octal) = 0xD106
Format:	SEX
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Description

Set the paging system to the 24-bit address mode instead of the 19-bit address mode.
(Creating a page table for 24-bit addresses)

Flags affected: SEXI (bit 13 of the STS register) cleared

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

Flags: SEXI

SHA

Shift A register
Every shift instruction places the last bit discarded in the multi-shift flag (M). M can be used as an input for the next shift instruction. M is bit 8 of the STS Register.

Opcode:

0154400 (octal) = 0xD900

Format:

SHA <shift_type> <shift_counter>

Mask:

1111_1001_1000_0000

Privilege:

User

Class:

shift_instructions

Bit Format

15	14	13	12	11	10	9		6	5	4	3	2	1	0	0
opcode					shift_type		register	mask6	shift_counter					SHR	

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
shift_type	10-9	enum	
register	7-8	enum	The register to shift
mask6	6	numeric	Always 0
SHR	0	assembler_flag	This a feature of the assembler. This mnemonic can be used to specify shift right, so that instead of calculating the 2's complement for the number of right shifts required, SHR can be used.
shift_counter	5-0	int6	The number of bits to shift the register. Bits 0-4 are the number of shifts. Bit 5=1, then shift right (max 32 times) Bit 5=0, then shift left (max 31 times)

Flags: M

SHD

Shift D register
Every shift instruction places the last bit discarded in the multi-shift flag (M). M can be used as an input for the next shift instruction. M is bit 8 of the STS Register.

Opcode:

154200 (octal) = 0xD880

Format:

SHD <shift_type> <shift_counter>

Mask:

1111_1001_1000_0000

Privilege:

User

Class:

shift_instructions

Bit Format

15	14	13	12	11	10	9		6	5	4	3	2	1	0	0
opcode					shift_type		register	mask6	shift_counter					SHR	

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
shift_type	10-9	enum	
register	7-8	enum	The register to shift
mask6	6	numeric	Always 0
SHR	0	assembler_flag	This a feature of the assembler. This mnemonic can be used to specify shift right, so that instead of calculating the 2's complement for the number of right shifts required, SHR can be used.
shift_counter	5-0	int6	The number of bits to shift the register. Bits 0-4 are the number of shifts. Bit 5=1, then shift right (max 32 times) Bit 5=0, then shift left (max 31 times)

Flags: M

SHDE

Decimal shift.

$(op2) \leftarrow (op1) \text{ shifted}$

This instruction is used to normalize operands for decimal operations.

The shift count determines whether the operand is shifted to the left or right:

(op2 - op1) positive then shift op1 to right

(op2 - op1) negative then shift op1 to left

If significant digits are lost by carrying out a left shift, an error is generated, directing the program counter to the instruction after the SHDE (the error return). If no errors occur this instruction is skipped.

The digits of the first operand are shifted and the result is placed in the second operand's memory location.

The number of places shifted is given by the difference in decimal position of the two operands. This normalizes the first operand (op1) to the second (op2) for decimal operations such as ADDD.

The sign of the normalized operand (op1) is as follows:

BCD Sign

14₈ +

15₈ -

An unsigned operand is converted to a plus unless bit 13 of the descriptor D2 is set, when the BCD equivalent of unsigned (17₈) is used.

The sign and digits of the first operand are checked before execution and any illegal digit codes reported.

If bit 10 of descriptor D2 (op2) is set the result is rounded, that is a 1 is added to the operand if the last digit shifted out of the field is ≥ 5 .

****Instruction sequence:****

SHDE

error handling instruction

next instruction after SHDE or after error handling routine

****Example:****

SHDE

JMP *10₈

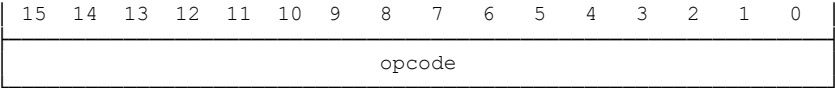
SAD 20₈

The SHDE instruction causes the program counter to skip the next instruction UNLESS an error has been generated, when the instruction immediately following the SHDE will handle the error in some way (in this example a jump is executed on error to (P) + 10₈)).

(* is the assembler mnemonic for the P register)

Opcode:	0140126 (octal) = 0xC056
Format:	SHDE
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

SHT

Shift T register

Every shift instruction places the last bit discarded in the multi-shift flag (M). M can be used as an input for the next shift instruction. M is bit 8 of the STS Register.

Opcode:	0154000 (octal) = 0xD800
Format:	SHT <shift_type> <shift_counter>
Mask:	1111_1001_1000_0000
Privilege:	User
Class:	shift_instructions

Bit Format

15	14	13	12	11	10	9		6	5	4	3	2	1	0	0
opcode					shift_type		register	mask6	shift_counter					SHR	

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
shift_type	10-9	enum	
register	7-8	enum	The register to shift
mask6	6	numeric	Always 0
SHR	0	assembler_flag	This a feature of the assembler. This mnemonic can be used to specify shift right, so that instead of calculating the 2's complement for the number of right shifts required, SHR can be used.
shift_counter	5-0	int6	The number of bits to shift the register. Bits 0-4 are the number of shifts. Bit 5=1, then shift right (max 32 times) Bit 5=0, then shift left (max 31 times)

Flags: M

SKP

The next instruction is skipped if a specified condition is true.

Opcode:

140000 (octal) = 0xC000

Format:

SKP <dr> <condition> <sr>

Mask:

1111_1000_1100_0000

Privilege:

User

Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode					condition			zeros		sr			dr		

Description

SKP can be qualified by eight different mnemonics which use the flags set by the following expression:

(dr) - (sr)

Four flags are affected by this calculation:

- S sign
- Z zero result (error)
- C carry
- O overflow

The condition codes are as follows:

<ascii-table>

bits	mnemonic	description	flag(s)	condition
10 9 8			if true	
0 0 0	EQL	Equal	Z = 1	
0 0 1	GEQ	Greater or equal to †	S = 0	
0 1 0	GRE	Greater or equal to * †	S + O = 0	
0 1 1	MGRE	Magnitude greater or equal *	C = 1	
1 0 0	UEQ	Unequal	Z = 0	
1 0 1	LSS	Less than †	S = 1	
1 1 0	LST	Less than * †	S + O = 1	
1 1 1	MLST	Magnitude less than *	C = 0	

</ascii-table>

* Overflow is taken care of
† Contents of sr and dr are treated as signed numbers

Note:
By swapping the sr and dr fields, these relationships can be tested:
> Greater than
<= Less than or equal

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
condition	10-8	enum	Condition to skip the next instruction
zeros	7-6	value	Must be 00
sr	5-3	src_register	The source register to be compared with the destination register
dr	2-0	dst_register	The destination register to be compared with the source register

Flags: S, Z, C, O

Examples

Skip next instruction if the D register contents equal that of the A register.

```
SKP DD EQL SL
```

Skip the next instruction if the contents of the A register are less than the B register contents.

OR

Skip the next instruction if the contents of the B register are greater than the A register contents.

```
SKP DB LSS SA
```

Skip the next instruction if the contents of the L register do not equal zero.

```
SKP DL UEQ
```

Skip the next instruction if the D register is less than zero.

```
SKP LSS SD
```

SRB

Store register block

Load the register block of the program level given in the instruction into the memory block pointed to by the X register.

If the instruction specifies the current program level, the P register points to the instruction following SRB.

Opcode:	152402 (octal) = 0xD502
Format:	SRB <level>
Mask:	1111_1111_1100_0000
Privilege:	Privileged
Class:	register_block

Bit Format

15	14	13	12	11	6	5	4	3	2	1	0
opcode					level				type		

Description

The register block of the specified program level is stored in the memory block pointed to by the X register. The mapping is as follows:

- (ea) ← P (level N)
- (ea + 1) ← X (level N)
- (ea + 2) ← T (level N)
- (ea + 3) ← A (level N)
- (ea + 4) ← D (level N)
- (ea + 5) ← L (level N)
- (ea + 6) ← STS (level N)
- (ea + 7) ← B (level N)

Where (ea) is the address in memory pointed to by the X register.

If the current program level is specified, the P register points to the instruction following SRB.

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
level	6-3	uint4	The level to load the register block to
type	2-0	enum	The type of register block function

Examples

SRB 100₈ (152702₈)

Store the register block of program level 8 in the memory block pointed to by X.

- (ea) ← P (level 8)
- (ea + 1) ← X (level 8)
- ↓ ↓
- (ea + 7) ← B (level 8)

SRB 100

STA

Store A register to memory location

$(EA) \leftarrow (A)$

Store the contents of the A register in the memory location pointed to by the effective address.

Opcode: 004000 (octal) = 0x0800
Format: STA <addressing_mode> <displacement>
Mask: 1111_1000_0000_0000
Privilege: User
Class: memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

STATX

Store A register

$(ea) \leftarrow (A)$

Store the contents of the A register in the memory location given by the effective address.

Opcode:	143304 (octal) = 0xC6C4
Format:	STATX <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

STD

Store double word

$(ea) \leftarrow (A)$
 $(ea) + 1 \leftarrow (D)$

Store the contents of the A register in the memory location pointed to by the effective address
Store the contents of the D register in the memory location pointed to by the effective address plus one.

Opcode:	020000 (octal) = 0x2000
Format:	STD <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15 14 13 12 11	10 9 8	7 6 5 4 3 2 1 0
	X I B	
opcode	addressing_mode	displacement

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

STD TX

Store double word

$(ea) \leftarrow (A)$
 $(ea) + 1 \leftarrow (D)$

Store the double word held in the A and D registers in the memory locations given by the effective address and the effective address plus one.

Opcode:	143306 (octal) = 0xC6C6
Format:	STD TX <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

STF

Store floating accumulator (TAD) to memory (ea)

Memory format:

- (ea) ← (T)
- (ea) + 1 ← (A)
- (ea) + 2 ← (D)

Store the contents of the floating accumulator (T,A and D registers) into the memory location pointed to by the effective address.

Opcode:

030000 (octal) = 0x3000

Format:

STF <addressing_mode> <disp>

Mask:

1111_1000_0000_0000

Privilege:

User

Class:

memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

STT

Store T register to memory location
 $(EA) \leftarrow (T)$

Store the contents of the T register in the memory location pointed to by the effective address.

Opcode:	010000 (octal) = 0x1000
Format:	STT <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15 14 13 12 11	10 9 8	7 6 5 4 3 2 1 0
	X I B	
opcode	addressing_mode	displacement

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

STX

Store X register to memory location
 $(EA) \leftarrow (X)$

Store the contents of the X register in the memory location pointed to by the effective address.

Opcode:	014000 (octal) = 0x1800
Format:	STX <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15 14 13 12 11	10 9 8	7 6 5 4 3 2 1 0
	X I B	
opcode	addressing_mode	displacement

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

STZ

Store zero to memory location
(EA) ← 0

Store the value zero in the memory location pointed to by the effective address.

Opcode:	000000 (octal) = 0x0000
Format:	STZ <addressing_mode> <displacement>
Mask:	1111_1000_0000_0000
Privilege:	User
Class:	memory_transfer

Bit Format

15 14 13 12 11	10 9 8	7 6 5 4 3 2 1 0
	X I B	
opcode	addressing_mode	displacement

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

STZTX

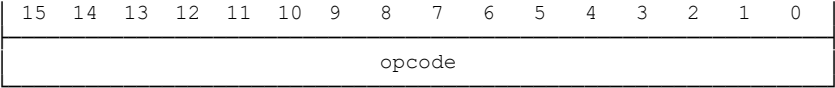
Store zero

$(ea) \leftarrow 000000_8$

Store zero in the memory location given by the effective address.

Opcode:	143305 (octal) = 0xC6C5
Format:	STZTX <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format



SUB

Subtract from A register
 $A \leftarrow A - (EL)$

Subtract the contents of the memory location pointed to by the effective address from the A register, leaving the result in A.

Opcode:

064000 (octal) = 0x6800

Format:

SUB <addressing_mode> <displacement>

Mask:

1111_1000_0000_0000

Privilege:

User

Class:

memory_transfer

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
					X	I	B								
opcode					addressing_mode			displacement							

Description

Flags affected:

- C=1: If a carry occurs from the signed bit positions of the adder.
- O=1, Q=1: If an overflow occurs, that is if the signs of the two operands are equal and the sign of the result is different.
- O=1, Q=0: If overflow does not exist, the dynamic overflow flag (O) is reset to 0 while the static overflow flag (Q) is left unchanged.

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode determines what type of operation occurs
addressing_mode	10-8	addressing_modes	These three bits give the addressing mode for the instruction
displacement	7-0	displacement	8-bit signed field gives the memory address displacement (2's complement notation giving a displacement range of -128 to 127 memory locations)

Flags: C, Q

SUBD

Subtract two decimal operands.

$(op1) \leftarrow (op1) - (op2)$

Subtract the second operand from the first operand, leaving the result in the location of the first operand.

If the first operand field is too short to contain all the significant digits of the result then decimal overflow occurs.

If bit 13 of D2 in the first operand is set, the sign of the result will be 17₈ (BCD unsigned).

Any empty operand, that is with a field length of zero, is treated as a positive zero.

A zero difference can have either a negative or positive sign.

****Instruction sequence:****

SUBD
error handling instruction
next instruction after SUBD or after error handling routine

****Example:****

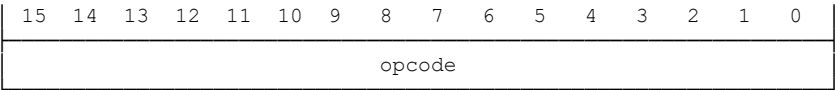
SUBD
JPL *30₈
ADD *15₈

The SUBD instruction causes the program counter to skip the next instruction UNLESS an error has been generated, when the instruction immediately following the SUBD will handle the error in some way (in this example, a jump is executed on error to a subroutine at (P) + 15₈)).

(* is the assembler mnemonic for the P register)

Opcode:	0140121 (octal) = 0xC051
Format:	SUBD
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

SWAP

Register swap

$dr \leftarrow sr$

Opcode:	144000 (octal) = 0xC800
Format:	SWAP <sub-instruction(s)> <source> <destination>
Mask:	1111_1111_1100_0000
Privilege:	User
Class:	register_logical

Bit Format

15	14	13	12	11	10	9	8	8	7	7	6	5	4	3	2	1	0
opcode					rad	ADC	AD1	CM2	CM1	CLD	source			destination			

Description

Optional sub-instructions:

- **CLD=1****:** Zero is used instead of the destination register as an operand (dr register contents are unchanged).
- **CM1=1****:** The 1's complement of the source register is used as an operand (sr register contents are unchanged).

Instruction combinations

SWAP <sr> <dr> $dr \leftarrow sr$
SWAP CLD <sr> <dr> $dr \leftarrow sr; sr \leftarrow 0$
SWAP CM1 <sr> <dr> $dr \leftarrow sr; sr \leftarrow dr$
SWAP CM1 CLD <sr> <dr> $dr \leftarrow sr; sr \leftarrow 0$

Operands

Name	Bits	Type	Description
opcode	15-11	opcode	The opcode for register inclusive OR
rad	10	constant	Logical operation flag
ADC	9	flag	Also addd old carry to destination
AD1	8	flag	Also add one to destination
CM1	7	flag	Use one's complement of source
CM2	8-7	flag	Two's complement (CM1 AD1)
CLD	6	flag	Clear destination before operation
source	5-3	enum	Source register (sr)
destination	2-0	enum	Destination register (dr)

Flags: C, O, Q

Examples

Exchange A and D register contents.

```
SWAP SA DD
```

Use zero as the destination operand. Exchange the A and X register contents. A register contents are zero, X register contents are the previous contents of A.

```
SWAP CLD SA DX
```

SZCB

Store zero in the core map-table bank (CMBNK).

(ea) <--- 0

ea = (B) + Δ = CMBNK entry

Δ: 3-bit displacement added to B included in the instruction opcode (bits 5-3)

Opcode:	140707 (octal) = 0xC1C7
Format:	SZCB <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

SZSB

Store zero in the segment-table bank (STBNK).

(ea) <--- 0

ea = (B) + Δ = STBNK entry

Δ: 3-bit displacement added to B included in the instruction opcode (bits 5-3)

Opcode:	140706 (octal) = 0xC1C6
Format:	SZSB <displacement>
Mask:	1111_1111_1100_0111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

TRA

Transfer to A register

The internal register given in the instruction is copied into the A register.

Opcode: 0150000 (octal) = 0xD000
Format: TRA <internal_register>
Mask: 1111_1111_1111_0000
Privilege: Privileged
Class:

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode												internal_register			

Operands

Name	Bits	Type	Description
opcode	15-4	opcode	The opcode determines what type of operation occurs
internal_register	3-0	enum	The CPU internal register

TRR

Transfer A to internal register
The internal register given in the instruction is loaded with the contents of the A register.

Opcode:
Format:
Mask:
Privilege:
Class:

0150100 (octal) = 0xD040
TRR <internal_register>
1111_1111_1111_0000
Privileged

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode												internal_register			

Operands

Name	Bits	Type	Description
opcode	15-4	opcode	The opcode determines what type of operation occurs
internal_register	3-0	enum	The CPU internal register

TSET

Test and set

- (T) points to the virtual memory word to be accessed
- (A) ← memory word addressed by T

The address given by the T register is a logical memory address. It is normally translated into a physical address using page tables (if memory management is on). The contents of the location addressed by T are simultaneously loaded into the A register as the location is written to with all 1s. The memory system is dedicated to this task and no other memory access is allowed during the operation. This can be used for processor synchronization.

The old content of the memory address is always read from the memory and never from cache. The all 1s' data word is never written to cache.

Opcode:	140123 (octal) = 0xC053
Format:	TSET
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

TSETP

Test and set physical memory word.

Opcode:	140516 (octal) = 0xC14E
Format:	TSETP
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Description

- (T) points to the physical memory bank to be accessed
- (X) points to the address within the bank
- (A) is loaded with the word

The contents of the location addressed by T and X are simultaneously loaded into the A register as the location is written to with all 1s. No other memory access is allowed during this operation.
The old content of the memory address is always read from the memory and never from cache. The all 1s' data word is never written to cache.
This instruction can be used for processor synchronization.

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

UPACK

Convert to ASCII.

(op2) ← (op1) in ASCII format

Convert the first operand from its BCD format to ASCII format, placing the result in the second operand location.

****Conversion process:****

The instruction carries out the following steps:

- 1. Checks the sign and digits of the operand (op1) are encoded as BCD digits. (Illegal codes generate an error code 2.)
- 2. Takes each BCD digit as the lower nibble of an equivalent ASCII digit. Sets the upper nibble of the ASCII byte (the zone) to 0011₂.
- 3. Converts the BCD sign of the operand to ASCII:

BCD	ASCII	Sign
0	53 ₈	+
1	55 ₈	-

- Note: If bit 13 of the descriptor D2 is set then the code 17₈ (BCD unsigned) is used.
- 4. Extends the second operand field (op2) with ASCII zeros (60₈) if the result is too small to fill the field. Reports overflow has occurred (error code 3), if the second operand field length is too short to contain the significant digits of the result (the remaining digits are ignored).

****Error code:****

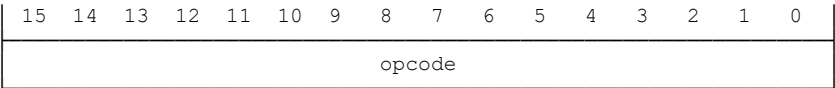
An error code is placed in bits 0 to 4 of the D register if an illegal code conversion is attempted or if overflow occurs. The first detected error will be reported.

- Error code: 2 illegal code†
- 3 overflow

† bit 15 of both the A and D registers point to the byte containing the illegal code.

Opcode:	0140125 (octal) = 0xC055
Format:	UPACK
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format



Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

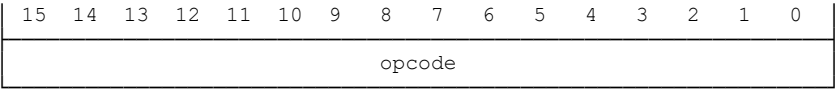
VERSN

**** ND-110/ND-120 ONLY****
Read ND-110 CPU version and installation number.

This instruction is used to read the version of ND-110 CPU installed. Three registers are loaded simultaneously with information in the following format:

Opcode:	140133 (octal) = 0xC05B
Format:	VERSN
Mask:	1111_1111_1111_1111
Privilege:	User
Class:	

Bit Format

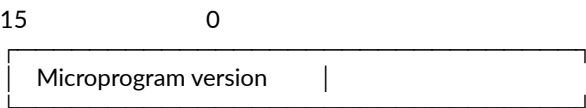


Description

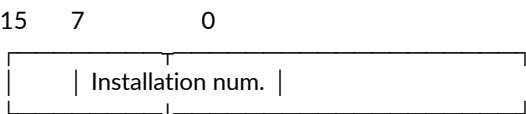
A register



T register



D register



The installation number of the CPU is 16 bytes long.

The VERSN instruction can only load one byte of the installation number into the D register each time it is executed. The A register is used to address the sixteen bytes.

To read a byte of the installation number the A register must be loaded with an installation byte address before the VERSN instruction is executed. The address of the byte to be read is given by the value of bits 8-11 in the A register (equivalent to bytes 0-16).

To read the complete installation number both the A register bit field must be incremented and the VERSN instruction executed sixteen times.

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

WAIT

When interrupt system off: halts the program and enters the operator's communication.
When interrupt system on: give up priority. If there are no interrupt requests on any level, the program on level zero is entered.

Opcode:	151000 (octal) = 0xD200
Format:	WAIT <wait_number>
Mask:	1111_1111_0000_0000
Privilege:	User
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode								wait_number							

Description

This operates as follows:

****If the interrupt system is OFF:****
The ND-110 stops with the program counter (P register) pointing to the instruction after the WAIT and the front panel RUN indicator is turned off. (To restart the system, type `!` on the console terminal.)

****If the interrupt system is ON:****
The ND-110 exits from the current program level (resetting the corresponding PID bit) and enters the program level with the highest priority, normally a program level lower than the one which executes the WAIT instruction. If there are no interrupt requests awaiting service, then program level 0 is entered.

****Note:****
A WAIT on program level 0 is ignored.

WAIT followed by a number less than 400₈ can be used to detect which location caused the program stop.

Operands

Name	Bits	Type	Description
opcode	15-8	opcode	The opcode determines what type of operation occurs
wait_number	7-0	numeric	Wait number

Examples

Wait

```
WAIT
```

Wait, but with a comment saying its WAIT #1

```
WAIT 1
```

WGLOB

Initialize global pointers.

- (T) = bank number of segment table (STBNK)
- (A) = start address within bank (STSRT)*
- (D) = bank number of core map table (CMBNK)

* must be divisible by 8

Opcode:	140500 (octal) = 0xC140
Format:	WGLOB
Mask:	1111_1111_1111_1111
Privilege:	Privileged
Class:	

Bit Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode															

Operands

Name	Bits	Type	Description
opcode	15-0	opcode	The opcode determines what type of operation occurs

ND-100

Processor Reference Manual

Norsk Data ND-100 16-bit Minicomputer

Generated by NDGen — 2026-04-05

Architecture

Word size: 16-bit
Byte order: Big Endian
Registers: A, D, B, L, T, X, STS, P
Addressing: Direct, Indirect, Indexed, Base-relative
Instructions: 153

Registers

Name	Size	Description
STS	16	Status register (user)
D	16	Data register
P	16	Program counter
B	16	Base register for addressing
L	16	Link register
A	16	Accumulator register
T	16	Temporary register
X	16	Index register

Instruction Index

Instructions are listed alphabetically.

Page	Mnemonic	Opcode	Description	Privilege
1	AAA	172400	Add argument to A	User
2	AAB	172000	Add argument to B	User
3	AAT	173000	Add argument to T	User
4	AAX	173400	Add argument to X	User
5	ADD	060000	Add to A register $A \leftarrow A + (EL)$ Add the contents of the memory loca...	User
6	ADDD	0140120	Add two decimal operands. $(op1) \leftarrow (op1) + (op2)$ Add the second op...	User
7	AND	070000	Logical AND to A register $A \leftarrow A \& (EA)$ Perform a bitwise AND opera...	User
8	BANC	177000	logical AND with bit complement $K \leftarrow K \& (B)0$ Perform a bitwise AN...	User
9	BAND	177200	Logical AND to K $K \leftarrow K \& (B)$	User
10	BFILL	0140130	Byte fill. Only the destination is used as an operand in this inst...	User
11	BLDA	176600	Load K $K \leftarrow \text{bit}$	User
12	BLDC	176400	Load K and complement $K \leftarrow (B) \text{ (complemented)}$	User
13	BORA	177600	Logical OR to K $K \leftarrow K (B)$	User
14	BORC	177400	Logical OR with bit complement $K \leftarrow K (B)0$	User
15	BSET	174000	Set specified bit in equal to specified condition	User
16	BSKP	175000	Skip next location if specified condition is true $P \leftarrow P+1$	User
17	BSTA	176200	Store and clear K $(B) \leftarrow K; K \leftarrow 0$	User
18	BSTC	176000	Store complement and set K to 1 $(B) \leftarrow K \text{ (complemented)}; K \leftarrow 1$	User
19	CHRENTPA GES	140303	Change page tables. The X register is used to address the current ...	Privileged

Page	Mnemonic	Opcode	Description	Privilege
20	CLEPT	140301	Clear page tables.	Privileged
21	CLEPU	140304	Clear page tables and collect PGU information.	Privileged
22	CLNREENT	140302	Clear non reentrant pages. The contents of the memory address at A...	Privileged
23	CLPT	140505	Clear segment from the page tables.	Privileged
24	CNREK	140504	Clear non reentrant pages (SINTRAN K only).	Privileged
25	COMD	0140122	Compare two decimal operands. (A) ← (op1) compared to (op2) (A) =...	User
27	COPY	0146100	Copy source to destination COPY is a compound mnemonic for RADD CL...	User
28	DEPO	150417	Deposit Store the contents of the T register in the physical memor...	Privileged
29	DNZ	152000	Denormalise Convert the floating number in TAD to a fixed poin...	User
30	ELEAV	0140137	Error leave stack. If an error occurs, leave the stack. This inst...	User
31	ENPT	140506	Enter segment in page tables.	Privileged
32	ENTR	0140135	Enter stack. This instruction saves the current stack pointer (B),...	User
33	EXAM	150416	Examine Load the contents of the physical memory location, pointed...	Privileged
34	EXIT	0146142	Return from subroutine This compound mnemonic represents the speci...	User
35	EXR	0140600	Execute instruction found in specified register The contents of th...	User
36	FAD	0100000	Add to floating point accumulator (A) ← (ea) + (T) (D) ← (ea +...	User
37	FDV	114000	Divide floating point accumulator The contents of the floating poi...	User
38	FMU	110000	Multiply floating point accumulator. The contents of the floating ...	User
39	FSB	104000	Subtract from floating point accumulator. The contents of two sequ...	User
40	GECO	0142700	GECO is a customer-specified instruction which appears to be include...	User
41	IDENT	0143600	Transfer IDENT code of interrupting device with highest priority on...	User
42	INIT	0140134	Initialize stack. Loads the addresses pointed to by B with the st...	User
43	INSPL	140502	Insert page in page list. (See Appendix B for a software descriptio...	Privileged
44	IOF	150401	Interrupt System OFF Disables the interrupt system. On IOF the ND-...	Privileged

Page	Mnemonic	Opcode	Description	Privilege
45	ION	150402	Interrupt System ON	User
46	IOT	160000	Enables the interrupt system. On ION the ND-i1... NORD-1 INSTRUCTION (DO NOT USE)	Privileged
47	IOX	0164000	Exchange information between I/O system and A register.	Privileged
48	IOXT	150415	Exchange information between I/O system and A register.	Privileged
49	IRR	153600	Inter Register Read A: = specified register on specified level	Privileged
50	IRW	153400	Inter Register Write Write A to specified register on specified level	Privileged
51	JAF	0131400	Condion: Jump if (A) != 0 (jump if A filled)	User
52	JAN	0130400	Condion: Jump if (A) < 0 (jump if A is negative)	User
53	JAP	0130000	Condion: Jump if (A) > 0 (jump if A positive)	User
54	JAZ	0131000	Condion: Jump if (A) == 0 (jump if A is zero)	User
55	JMP	0124000	Jump - Unconditional jump to specified address The next instruction...	User
56	JNC	0132400	Increment X and jump if X is negative	User
57	JPC	0132000	Increment X and jump if X is positive	User
58	JPL	134000	Jump if Plus - Jump to specified address if the result of the last ...	User
59	JXN	0133400	Condion: Jump if (X) < 0 (jump if X negative)	User
60	JXZ	0133000	Condion: Jump if (X) == 0 (jump if X is zero)	User
61	LACB	140702	Load the A register from the core map-table bank (CMBNK). (A) <---...	Privileged
62	LASB	140700	Load the A register with the contents of the segment-table bank (ST...	Privileged
63	LBIT	140510	Load single bit accumulator (K) with logical memory bit. (X) point...	Privileged
64	LBITP	140511	Load single bit accumulator (K) with physical memory bit. (T) poin...	Privileged
65	LBYT	142200	Load byte from memory to A register Addressing: $EL = (T) + (X)/2$ ~...	User
66	LBYTP	140514	Load the A register with a byte from physical memory. (D) points t...	Privileged
67	LDA	044000	Load A register Load the contents of the memory location pointed t...	User
68	LDATX	143300	Load A register (A) ← (ea) Load the contents of the physical memo...	Privileged
69	LDBTX	143303	Load B register (B) ← 177000 ₈ OR (2(ea)) Load the contents of the...	Privileged
70	LDD	024000	Load double word	User

Page	Mnemonic	Opcode	Description	Privilege
			$A \leftarrow (ea)$ $D \leftarrow (ea) + 1$ Load the contents of th...	
71	LDDTX	143302	Load double word $(A) \leftarrow (ea)$ $(D) \leftarrow (ea) + 1$	Privileged
72	LDF	034000	Load the contents of t... Load floating accumulator (TAD) from memory (FW) Memory format: (T...	User
73	LDT	050000	Load T register $(T) \leftarrow (ea)$	User
74	LDX	054000	Load the contents of the memory locati... Load X register $(X) \leftarrow (ea)$	User
75	LDXTX	143301	Load the contents of the memory locati... Load X register $(X) \leftarrow (ea)$	Privileged
76	LEAVE	0140136	Load the contents of the physical memo... Leave stack.	User
77	LRB	152600	This instruction saves the previous stack pointer in ... Load register block	Privileged
79	LWCS	143500	Load the contents of a memory block pointed to... Writable Control Store Instruction	Privileged
80	LXCB	140705	This instruction is PRIVILEGED ... Load the X register from the core table bank (CMBNK).	Privileged
81	LXSB	140704	$(X) \leftarrow (ea)$ Load the X register from the segment table bank (STBNK).	Privileged
82	MCL	0150200	$(X) \leftarrow (ea)$ Masked clear	Privileged
83	MIN	040000	The A register is used as a mask to clear bits within... Memory increment and skip next instruction if zero (EA): $= (EA) + 1$...	User
84	MIX3	143200	Multiply index by 3. $(X) \leftarrow [(A)-1] * 3$	User
85	MON	153000	Take the contents of the A... The MON instruction is used in special different contexts when runn...	User
86	MOVB	0140131	Move byte.	User
87	MOVBF	0140132	This instruction moves a block of bytes from the memory... Move bytes forward.	User
88	MOVEW	0143100	This instruction moves a block of bytes from t... Word block move.	User
90	MPY	120000	This instruction moves a block of words from one ... Multiply integer $A \leftarrow A * (EL)$	User
91	MST	0150300	Multiply the contents of the memory ... Masked set	Privileged

Page	Mnemonic	Opcode	Description	Privilege
92	NLZ	0151400	The A register is used as a mask to set bits within the...	User
			Normalize	
			Convert the number in A to a floating number in TAD	
93	OPCOM	150400	Operator Communication	Privileged
94	ORA	074000	This instruction is PRIVILEGED and only ava...	User
			Logical inclusive OR to A register $A \leftarrow A \mid (EA)$	
95	PACK	0140124	Perform a bitwise ...	User
			Convert to BCD.	
			$(op2) \leftarrow (op1)$ in BCD format	
96	PIOF	150405	Convert the first ope...	Privileged
			Memory management and interrupt system OFF	
97	PION	150412	Disables both the memor...	User
			Memory management and interrupt system ON	
98	POF	150404	Enable both the memory m...	Privileged
			Memory management OFF	
99	PON	150410	Disable memory management system. The next i...	User
			Memory management ON	
100	RADD	0146000	Enable memory management system. The next ins...	User
			Add source to destination (dr): $= (dr) + (sr)$	
102	RAND	144400	Logical AND to destination (dr): $= (dr) \& (sr)$	User
103	RCLR	0146100	Register clear	User
104	RDCR	146200	Register decrement	User
			$dr \leftarrow dr - 1$	
105	RDIV	141600	RDCR is a compound mnemonic for RA...	User
106	RDUS	140127	Divide double accumulator with source; quotient in A, remainder in ...	Privileged
			Read a word without using cache	
107	RDUSP	140517	(T) points to the virtual memory w...	Privileged
			Read a physical memory word without using cache.	
108	REMP	140503	(T) points to the...	Privileged
			Remove page from page list.	
109	REPT	140507	Enter reentrant segment in page tables. (See Appendix B for a softw...	Privileged
110	REX	0150407	Reset extended address mode	Privileged
111	REXO	0145000	Logical exclusive OR	User
			(dr): $= (dr) \oplus (sr)$	
112	RGLOB	140501	Examine global pointers.	Privileged
			(T) <--- bank number of segment table (ST...	
113	RINC	146400	Register increment	User
			$dr \leftarrow dr + 1$	
			RINC is a compound mnemonic for RA...	

Page	Mnemonic	Opcode	Description	Privilege
114	RMPY	141200	Multiply source with destination; result in double accumulator $AD \leftarrow \dots$	User
115	RORA	145400	OR register $dr \leftarrow dr \text{ OR } sr$	User
116	RSUB	146600	Register subtract $dr \leftarrow dr - sr$ RSUB is a compound mnemonic for RA...	User
117	SAA	170400	Set argument to A	User
118	SAB	170000	Set argument to B	User
119	SACB	140703	Store the A register in the core map table bank (CMBNK). $(ea) \leftarrow \dots$	Privileged
120	SAD	0154600	Shift A and D registers connected Every shift instruction places th...	User
121	SASB	140701	Store the A register contents in the segment table bank (STBNK). (...	Privileged
122	SAT	171000	Set argument to T	User
123	SAX	171400	Set argument to X	User
124	SBIT	140512	Store the single bit accumulator (K) in a logical memory bit. (X) ...	Privileged
125	SBITP	140513	Store the single bit accumulator (K) in a physical memory bit. (T)...	Privileged
126	SBYT	142600	Store byte from A register to memory Addressing: $EL = (T) + (X)/2$ ~...	User
127	SBYTP	140515	Store a byte in physical memory. (D) points to the bank number con...	Privileged
128	SETPT	140300	Set page tables.	Privileged
129	SEX	150406	Set extended address mode	Privileged
130	SHA	0154400	Shift A register Every shift instruction places the last bit discar...	User
131	SHD	154200	Shift D register Every shift instruction places the last bit discar...	User
132	SHDE	0140126	Decimal shift. $(op2) \leftarrow (op1) \text{ shifted}$ This instruction is used to ...	User
134	SHT	0154000	Shift T register Every shift instruction places the last bit discar...	User
135	SKP	140000	The next instruction is skipped if a specified condition is true.	User
137	SRB	152402	Store register block Load the register block of the program level ...	Privileged
139	STA	004000	Store A register to memory location $(EA) \leftarrow (A)$ Store the contents...	User
140	STATX	143304	Store A register $(ea) \leftarrow (A)$ Store the contents of the A register ...	Privileged
141	STD	020000	Store double word	User

Page	Mnemonic	Opcode	Description	Privilege
			$(ea) \leftarrow (A)$ $(ea) + 1 \leftarrow (D)$ Store the cont...	
142	STDTX	143306	Store double word	Privileged
			$(ea) \leftarrow (A)$ $(ea) + 1 \leftarrow (D)$ Store the double word...	
143	STF	030000	Store floating accumulator (TAD) to memory (ea) Memory format: (ea...	User
144	STT	010000	Store T register to memory location $(EA) \leftarrow (T)$	User
145	STX	014000	Store the contents ... Store X register to memory location $(EA) \leftarrow (X)$	User
146	STZ	000000	Store the contents... Store zero to memory location $(EA) \leftarrow 0$	User
147	STZTX	143305	Store the value zero in the... Store zero $(ea) \leftarrow 000000_8$	Privileged
148	SUB	064000	Store zero in the memory location given... Subtract from A register $A \leftarrow A - (EL)$	User
149	SUBD	0140121	Subtract the contents of the... Subtract two decimal operands. $(op1) \leftarrow (op1) - (op2)$	User
150	SWAP	144000	Subtract the... Register swap $dr \leftarrow sr$	User
151	SZCB	140707	Store zero in the core map-table bank (CMBNK). $(ea) <--- 0$	Privileged
152	SZSB	140706	$ea = (...)$ Store zero in the segment-table bank (STBNK). $(ea) <--- 0$	Privileged
153	TRA	0150000	$ea = (B...$ Transfer to A register The internal register given in the instructi...	Privileged
154	TRR	0150100	Transfer A to internal register The internal register given in the ...	Privileged
155	TSET	140123	Test and set (T) points to the virtual memory word to be accessed ...	Privileged
156	TSETP	140516	Test and set physical memory word.	Privileged
157	UPACK	0140125	Convert to ASCII. $(op2) \leftarrow (op1)$ in ASCII format Convert the first...	User

Page	Mnemonic	Opcode	Description	Privilege
158	VERSN	140133	** ND-110/ND-120 ONLY** Read ND-110 CPU version and installation nu...	User
160	WAIT	151000	When interrupt system off: halts the program and enters the operato...	User
161	WGLOB	140500	Initialize global pointers.	Privileged

(T) = bank number of segment table (ST...